

# Dynamic scheduling for flexible job shop under machine breakdown using Improved Double Deep Q-network

Rui Wu<sup>a,b</sup>, Jianxin Zheng<sup>a,b</sup>, Xixing Li<sup>a,b,\*</sup>, Hongtao Tang<sup>c</sup>, Xi Vincent Wang<sup>d</sup>, Yibing Li<sup>c</sup>

<sup>a</sup> Hubei Key Laboratory of Modern Manufacturing and Quality Engineering, Hubei University of Technology, Wuhan, 430068, China

<sup>b</sup> School of Mechanical Engineering, Hubei University of Technology, Wuhan, 430068, China

<sup>c</sup> School of Mechanical and Electronic Engineering, Wuhan University of Technology, Wuhan, 430070, China

<sup>d</sup> Department of Production Engineering, KTH Royal Institute of Technology, Sweden

## ARTICLE INFO

### Keywords:

Dynamic flexible job shop scheduling  
Machine breakdown  
Dispatching rules  
Improved Double Deep Q-Network

## ABSTRACT

Machine breakdown is an ordinary disruption in job shop environments, upon a machine breakdown, operations in progress or pending commencement need to be rescheduled. This paper addresses the Dynamic Flexible Job Shop Scheduling Problem (DFJSP) under machine breakdown, with the objective of minimizing total tardiness, and an Improved Double Deep Q-Network (IDDPQN) for dynamic selection of dispatch rules is proposed to solve DFJSP. In IDDPQN, five state features are formulated and tailored for the dynamic job shop scheduling environment, 12 distinct dispatch rules are employed as actions, the structure of the experience replay buffer is redesigned and a unique reward function is proposed, unique iteration method and dueling network architecture is designed. To validate the efficacy and versatility of the IDDPQN, we conducted numerical experiments on numerous instances with varying production configurations. The results unequivocally affirm the superiority and generality of the IDDPQN when compared to individual composite rules and even the meta-heuristic optimization algorithms in isolation.

## 1. Introduction

In contemporary manufacturing, the production landscapes encompassing semiconductor manufacturing processes, building materials manufacturing, and mechanical manufacturing systems can be abstractly distilled into the realm of flexible job shop scheduling problems. Recognized as an NP-hard problem (Li, Gong, Wang, Lu, & Dong, 2024), FJSP has garnered extensive utilization within modern industries. Its intricate nature and profound interconnection with production environment have spurred comprehensive investigations into FJSP modeling and solution methodologies (Dauzère-Pérès, Ding, Shen, & Tamssaouet, 2023). Compared to the established NP-hard classic Job Shop Scheduling problem (JSP), the flexibility of machines in FJSP allows for a more refined equilibrium in their workload distribution. The variability of processing times for job operations across different machines significantly influences the scheduling scheme.

Most existing methods addressing FJSP have presumed a static manufacturing environment where the job shop's status is entirely predetermined. However, in today's complex and continually evolving landscape of manufacturing systems, the exploration of dynamic disturbances has emerged as a crucial area of investigation. These disruptions encompass

various elements, including the introduction of jobs, changes or cancellations in orders, machine breakdown, and fluctuations in the processing duration for operations (Liu & Huang, 2023). Such disturbances significantly disrupt the practical execution of static plans, leading to a notable decline in production efficiency compared to their projected outcomes. Thus, it is imperative to develop online scheduling algorithms specifically designed for DFJSP that efficiently handle real-time dynamic events.

The domain of dynamic scheduling challenges has been a compelling subject, attracting extensive attention and research efforts from both academic circles and industry domains over the past few decades. Heuristic algorithms and meta-heuristic algorithms are fundamental methodologies for addressing this intricate issue (Destouet, Tlahig, Bettayeb, & Mazari, 2023). In the realm of heuristic algorithms, research primarily revolves around purposefully designed dispatch rules, while meta-heuristic algorithms encompass various methodologies such as Genetic Algorithm (GA) (Song et al., 2024; Yang, Shen, & Han, 2024), Tabu Search (TS) (Peng, Lü, & Cheng, 2015; Shen, Dauzère-Pérès, & Neufeld, 2018), Ant Colony Optimization (ACO) (Fu et al., 2024; Wang, Cai, Li, & Liu, 2017), Differential Evolution algorithm (DE) (Guo et al., 2024b; Liao, Gong, & Li, 2023) and Particle Swarm Optimization (PSO)

\* Corresponding author.

E-mail addresses: [wurui@hbut.edu.cn](mailto:wurui@hbut.edu.cn) (R. Wu), [102210164@hbut.edu.cn](mailto:102210164@hbut.edu.cn) (J. Zheng), [lixixing@hbut.edu.cn](mailto:lixixing@hbut.edu.cn) (X. Li), [310836@whut.edu.cn](mailto:310836@whut.edu.cn) (H. Tang), [wangxi@kth.se](mailto:wangxi@kth.se) (X.V. Wang), [ahlyb@whut.edu.cn](mailto:ahlyb@whut.edu.cn) (Y. Li).

<https://doi.org/10.1016/j.eswa.2025.128280>

Received 23 September 2024; Received in revised form 11 May 2025; Accepted 20 May 2025

Available online 24 May 2025

0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(Liu, Fang, Zhan, & Zhang, 2023; Zhao, Shi, & Liu, 2023). In recent studies, Genetic Programming (GP) has also become a focus of research (Branke, Nguyen, Pickardt, & Zhang, 2016; Zhang, Nguyen, Mei, & Zhang, 2021). Nguyen, Mei, and Zhang (2017) proposed a unified framework to design production scheduling heuristics using genetic programming automatically. Lin, Zhu, and Gao (2020) proposed a genetic programming hyper-heuristic method for multi-skill resource-constrained project scheduling problems. Ten simple heuristic rules were designed to construct a set of low-level heuristics, and genetic programming was used as a high-level strategy to manage the low-level heuristics flexibly. In the study of Guo, Liu, Wang, and Zhuang (2024a), a GP used for dynamic flexible shop scheduling problems was proposed. They adopted an improved genetic programming hyper-heuristic method with completion time, delay time, and reconfiguration time as optimization objectives. Chen, Bai, Qu, Dong, and Jin (2024) introduced an integrated hyper-heuristic approach that combines deep reinforcement learning with genetic programming. This method employs reinforcement learning agents to manage a collection of automatically generated low-level heuristics aimed at dynamically scheduling container port trucks. The research highlights the importance of efficient truck scheduling in optimizing the operations of container terminals in dynamic and complex situations. Xu, Mei, Zhang, and Zhang (2024) proposed a novel approach combining genetic programming and reinforcement learning to address the problem of the dynamic flexible job shop scheduling problem. Traditional scheduling methods often struggle to adapt to rapidly changing environments. In response, the authors proposed using GP to develop dispatch rules that replace manually designed ones. These heuristics are further enhanced through RL, enabling them to optimize and adjust based on real-time feedback from the environment. The experimental results indicate that this new method outperforms traditional dispatch rules and reinforcement learning.

Relying solely on a singular dispatch rule lacks a multifaceted computational selection process, thus ensuring optimal time efficiency. However, this single-rule approach poses challenges in ensuring the best result, consequently complicating the achievement of optimization objectives. In parallel, decision-makers encounter a challenge in pinpointing the optimal rule precisely at any given moment due to the varying suitability of different rules across diverse scenarios. This complexity leads to a proliferation of schedule results, posing obstacles to their effective application in addressing the intricacies of the DFJSP. Meta-heuristic algorithms consistently divide dynamic scheduling problems into a sequence of static sub-problems, which are individually solved using sophisticated optimization algorithms. Although this approach yields superior solution quality, its time-intensive nature renders it unsuitable for meeting real-time dynamic scheduling demands. Hence, the primary research objective of this paper is to strike a balance between timeliness and scheduling performance by judiciously selecting the most appropriate dispatch rule at each rescheduling juncture.

To ascertain the most fitting dispatch rule at each decision juncture, a pivotal aspect of DFJSP involves the modeling of the scheduling problem as a Markov Decision Process (MDP) (Lei et al., 2022). The MDP constitutes a mathematical framework for outlining decision-making scenarios encountered by an agent functioning within an environment that exhibits Markov attributes. It is signifying that subsequent actions, based on the current state, remain unaffected by past sequences of events or actions, encapsulating the fundamental principle of state transition (Zhao, Zong, Tian, Zhang, & You, 2019). In the domain of deep reinforcement learning (DRL), the orchestration of dynamic task allocation decisions, resource distribution, and the determination of optimal actions based on environmental states are encapsulated within the framework of MDP to maximize long-term rewards (Shakya, Pillai, & Chakrabarty, 2023). Nevertheless, accurately determining transition probabilities for each state-action pair in today's intricate production landscapes presents a substantial challenge within MDP. This limitation significantly hampers the practical application of dynamic programming in industrial production. Furthermore, relying on alternative methods, such as heuris-

tic dispatch rules, may not consistently yield high-quality scheduling solutions in complex production settings. Similarly, meta-heuristic approaches might be time-consuming and struggle to adapt to dynamic event interference promptly. Consequently, an increasing number of scholars are exploring novel approaches to address these formidable challenges.

Advancing research increasingly relies on reinforcement learning (RL) to navigate the intricacies of production environments. RL effectively manages MDP, leading to noteworthy progress and achievements. Due to its capacity to select optimal actions at crucial decision points and promptly adapt to dynamic events in real-time, a plethora of RL-based methodologies have found application across various categories of dynamic scheduling challenges. Wang (2020) address job shop scheduling by utilizing a Weighted Q-learning algorithm, which integrated clustering techniques and dynamic search strategies. The primary focus is on minimizing earliness and tardiness punishment. Han, Guo, and Su (2019) utilize reinforcement Learning methodologies to tackle the hybrid flow shop scheduling problem to minimize the makespan. Shahrabi, Adibi, and Mahootchi (2017) address the dynamic job shop scheduling problem incorporating random job arrivals and machine breakdown. They utilize a variable neighborhood search technique, dynamically adjusting its parameters using RL to minimize the average flow time. Wang, Sarker, Li, and Li (2021) apply dual Q-learning to address an assembly job shop scheduling problem characterized by uncertain assembly times, aiming to minimize the overall weighted earliness penalty and completion time costs. Yan, Wang, and Wu (2022) employ the double-layer Q-learning algorithm to address a digital twin-enabled integrated optimization problem involving the FJSP and flexible preventive maintenance.

In the majority of the previously mentioned RL-based methodologies, Q-learning emerges as a prevalent choice, particularly suited for problems characterized by a discrete and finite state space. However, when dealing with continuous state problems, the necessity to maintain a Q-table lookup and dummyTXdummy- reduce computational complexity often leads to a compromise in precision in modeling. Hence, an increasing number of scholars have begun to adopt deep reinforcement learning algorithms to address complex dynamic problems. Gui, Tang, Zhu, Zhang, and Zhang (2023) propose a hierarchical algorithm for the flexible job shop scheduling problem with job insertion, based on the DRL. Hammami, Lardeux, Atidel, and Jridi (2024) apply a DRL-based approach to solve the real-time job shop scheduling problem facing unexpected job arrivals. Zhao, Fan, Zhang, Shen, and Zhuang (2024) design a DRL-based reactive scheduling method, proximal policy optimization with attention-based policy network, which is proposed to make real-time decisions for the random job arrivals. Gan, Zuo, Yang, Zhang, and Tao (2024) use the DRL to address the machine breakdown for dual-objective job shop. A bi-objective DFJSP model with the objective of maximum completion time and robustness is established. And an effective two-stage algorithm based on convolutional neural network is proposed to solve the FJSP with machine breakdown (Zhang et al., 2022).

Drawing upon the above literature review, the prevailing focus within the current research landscape revolves around single-machine scheduling, flow shop scheduling, and dynamic job shop scheduling. Research on applying deep reinforcement learning algorithms to solve flexible job shop scheduling problems under machine breakdown conditions is still in its early stages.

The ascendancy of DRL methods over traditional meta-heuristic approaches, concerning solution quality and generalization, remains an aspect yet to be definitively clarified. Illustratively, in the research conducted by Luo (2020), the introduced DRL demonstrated only superiority when compared solely to dispatch rules and Q-learning.

Machine breakdown is an unavoidable processing disruption in manufacturing enterprises and often result in direct economic losses. We need a quick response to avoid job delays when a machine breakdown occurs. In consideration of the aforementioned factors, this study has

proposed an enhanced DRL approach tailored to address the challenges of DFJSP with machine breakdown. The primary objective is to minimize the total job tardiness, thereby facilitating efficient optimization for DFJSP. The contributions of this work are summarized as follows.

1. **Problem Extension:** The DFJSP is studied based on the manufacturing scheduling problem. It considers the FJSP with different processing times of random machine breakdown. Machine breakdown is a common occurrence exists in various discrete processing and manufacturing environments, such as the processing and manufacturing of aviation, aerospace, ships and auto parts. It often causes irreparable damage to the scheduling plan.
2. **Algorithm Design:** An improved algorithm IDDQN is proposed, which is based on the DDQN-based framework. Five state features are formulated and tailored for the job shop scheduling environment and employ 12 distinct combinations of dispatch rules. The objective is to derive efficient solutions for the present scheduling challenges. The algorithm uses a unique iteration method and imports a dueling network architecture to achieve high-quality solutions. The experience pool structure is reset and combined with the reward function to improve performance. When solving different scales of DFJSP, it has strong autonomy and can decide the optimal schedule.
3. **Compared Experiment:** During the experimental session, the twelve dispatch rules of action space are compared to verify the effectiveness of the proposed algorithm, which can select appropriate dispatch rules for various production situations. We also compared the DDQN to ensure the improvement effectiveness. Additionally, DRL is need to compared with meta-heuristic algorithms and IDDQN has achieved better results.

The subsequent sections are organized as follows. Section 2 introduces the problem statement and the Mixed Integer Linear Programming(MILP) model of DFJSP. Section 3 introduces our approach in detail. Section 4 unfolds the experimental results and engages in a thorough discussion of the findings, and finally, Section 5 concludes this paper.

## 2. Problem description and MILP model

### 2.1. Problem description

The decision problem formulation for rescheduling in the event of a machine breakdown in a flexible job shop is articulated as follows: A set of  $n$  jobs awaiting processing, denoted as  $J = \{J_1, J_2, \dots, J_n\}$ , is allocated to  $m$  machines, where  $M = \{M_1, M_2, \dots, M_m\}$ . Each job  $J_i$ , consists of  $n_i$  operations, with  $O_{i,j}$  representing the  $j$ th operation of job  $J_i$ . Each operation, denoted as  $O_{i,j}$ , possesses the flexibility to be processed on any machine  $M_k$  within the compatible machine set  $M_{i,j}$  (where  $M_{i,j} \subseteq M$ ), and the processing time for each operation exhibits variability across the optional machines. The duration required for the processing operation  $O_{i,j}$  on machine  $M_k$  is denoted as  $P_{i,j,k}$ . The stipulated delivery deadline for job  $J_i$  is represented by  $D_i$ .  $C_{i,j}$  denotes the actual completion time of operation  $O_{i,j}$ . The primary optimization objective of this research is to minimize the total tardiness across all jobs, promptly respond to disturbances caused by machine breakdown.

In the event of a machine  $M_k$ , currently processing operation  $O_{i,j}$ , encountering a failure at time  $t$ , three plausible scenarios merit consideration: (1) the faulty machine is actively engaged in processing an operation; (2) the faulty machine is idle, but there exist subsequent processing tasks necessitating allocation; (3) the faulty machine is idle, and there are no ensuing processing tasks. It is noteworthy that the third scenario, which does not necessitate production rescheduling, falls outside the scope of this study. Therefore, in this research, we focus on the first two scenarios, which necessitate rescheduling operations in response to machine breakdown.

Upon the machine breakdown, the completed operations remain unaffected. The processing operation on the faulty machine is promptly

**Table 1**

Notations description.

| Notations     | Description                                                                                                                                                                               |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $i, h$        | The index of job                                                                                                                                                                          |
| $j, g$        | The index of operation                                                                                                                                                                    |
| $n$           | Total number of jobs                                                                                                                                                                      |
| $m$           | Total number of machines                                                                                                                                                                  |
| $J_i$         | $i$ -th job                                                                                                                                                                               |
| $n_i$         | The number of operations for $J_i$                                                                                                                                                        |
| $M_k$         | $k$ -th machine                                                                                                                                                                           |
| $O_{i,j}$     | The $j$ -th operation of $J_i$                                                                                                                                                            |
| $S_{i,j}$     | The start processing time of operation $O_{i,j}$                                                                                                                                          |
| $C_{i,j}$     | The completion time of operation $O_{i,j}$                                                                                                                                                |
| $C_i$         | The completion time of $J_i$                                                                                                                                                              |
| $P_{i,j,k}$   | The processing time of operation $O_{i,j}$ on machine $M_k$                                                                                                                               |
| $D_i$         | The due date of $J_i$                                                                                                                                                                     |
| $X_{i,j,k}$   | If operation $O_{i,j}$ is processed on $M_k$ , $X_{i,j,k} = 1$ ; else $X_{i,j,k} = 0$                                                                                                     |
| $N_{i,j,h,g}$ | If $O_{h,g}$ is the previous operation adjacent to $O_{i,j}$ , $N_{i,j,h,g} = 1$ ; If $O_{h,g}$ is the next operation adjacent to $O_{i,j}$ , $N_{i,j,h,g} = -1$ ; else $N_{i,j,h,g} = 0$ |
| $t$           | Scheduling time                                                                                                                                                                           |
| $M_{i,j}$     | The available machine set $M_1, \dots, M_m$ for the $O_{i,j}$                                                                                                                             |
| $M_k(t)$      | Machine status at time $t$ , if the machine breakdown $M_k(t) = -1$ , else $M_k(t) = 1$                                                                                                   |

disrupted, halting the processing and needs to be reprocessed. Simultaneously, the operations currently being processed on normal machines remain unchanged in their state, but the subsequent processing schedules may be subject to adjustment.

To facilitate research on the DFJSP, we make the following assumptions.

1. Initially, all jobs are available.
2. Each machine is incapable of simultaneously processing two operations.
3. The processing time of each operation on the available machines is assumed to be known
4. Failures are assumed to occur on only one machine at same time.

The notations employed for problem formulation are delineated below in Table 1.

### 2.2. MILP Model

In order to accurately describe the DFJSP, a MILP is established. In the context of the DFJSP, the primary goal is to minimize tardiness. Based on the notations above, the DFJSP addressed in this paper can be described mathematically as follows.

In the job shop of scheduling, refers to the delay incurred when the completion time of a job exceeds its designated delivery date. It is calculated as the sum of delays for all jobs. Moreover, the total tardiness  $D$  objective is described as

$$\min D = \sum_{i=1}^n \max(C_i - D_i, 0) \quad (1)$$

Among them,  $C_i$  is the last operation completion time of  $J_i$ .

$$C_i = C_{i,n_i}, \quad \forall i \in \{1, 2, \dots, n\} \quad (2)$$

And the completion time of operation  $O_{i,j}$  can be calculated as

$$C_{i,j} = S_{i,j} + \sum_{k=1}^m P_{i,j,k} \times X_{i,j,k}, \quad (3)$$

$$\forall i \in \{1, 2, \dots, n\}, j \in \{1, 2, \dots, n_i\}$$

Each operation can only be executed on one machine at the same time. This can be described as

$$\sum_{k \in M_{i,j}} X_{i,j,k} = 1, \quad \forall i \in \{1, 2, \dots, n\}, j \in \{1, 2, \dots, n_i\} \quad (4)$$

The process sequence constraint can be described as

$$S_{i,j} - C_{i,j-1} > 0, \quad \forall i \in \{1, 2, \dots, n\}, j \in \{1, 2, \dots, n_i\} \quad (5)$$

The operation can only be processed on normal machines. This can be described as

$$X_{i,j,k} \times M_k(t) \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, j \in \{1, 2, \dots, n_i\}, k \in \{1, 2, \dots, m\}, 0 \leq t \leq \max(C_i) \quad (6)$$

Each machine can process only one operation at a time, that operation must be processed sequentially. This can be described as

$$(S_{h,g} - C_{i,j})X_{i,j,k}X_{h,g,k}(N_{i,j,h,g} + 1) + (S_{i,j} - C_{h,g})X_{i,j,k}X_{h,g,k}(1 - N_{i,j,h,g}) \geq 0, \quad \forall i, h \in \{1, 2, \dots, n\}, j, g \in \{1, 2, \dots, n_i\}, k \in \{1, 2, \dots, m\} \quad (7)$$

The stochastic failure time  $t$  for machines follows a Weibull distribution (Das, 2008). Considering the impact of aging on machines, the probability of machine breakdown is directly associated with time. For this study, specific values are chosen for the shape parameter  $k$  and the scale parameter  $\lambda$  set at 5 and 1, respectively, aligning with the distinctive characteristics of the DFJSP.

$$MS_k(t) = \begin{cases} \frac{k}{\lambda} (\frac{t}{\lambda})^{k-1} e^{-(\frac{t}{\lambda})^k}, & t \geq 0; \\ 0, & t < 0. \end{cases} \quad (8)$$

### 3. Our approach: IDDQN

DRL emerges as a cutting-edge machine learning paradigm in the domain of artificial intelligence (Sahu, Mokhade, & Bokde, 2023). Its distinctive feature lies in integrating deep neural networks with RL techniques (Wang et al., 2022). Through iterative interactions with the environment, DRL engages in real-time learning processes to adapt and refine its behavioral strategy, with the ultimate goal of maximizing long-term rewards (Li, Assis, Yan, & Simeonidou, 2022). In contrast to conventional RL approaches, DRL employs deep neural networks for both feature extraction and prediction of the probability distribution of optimal actions. This approach informs the decision-making process based on the current state, enabling more effective integration of learned features to action selection (Zhang, Jiang, & Zhang, 2024). Furthermore, DRL can enhance training efficiency and stability by incorporating techniques such as memory replay and double Q-networks (Zheng et al., 2022). Fig. 1 clarifies the procedural aspects of the basic DDQN. At the core of DRL lies a triad consisting of the agent, neural network, and environment. In this paradigm, the environment is conceptualized as a system with a multitude of intricate states. At each time step  $t$ , the agent receives the current state  $s$  and, guided by both the current state and a selection of available actions, chooses to execute a specific action.

Applying DDQN to tackle the DFJSP, the primary challenge lies in formulating a strategy to navigate the complex environment adeptly. This necessitates defining appropriate state features that subsequently serve as inputs to the neural network. The subsequent task involves developing a methodology for selecting appropriate actions based on the current environment, with the overarching objective of achieving maximal long-term rewards. In the realm of RL-based scheduling methodologies, state features are conventionally construed as indicators reflecting the production state. These indicators may encompass metrics such

as the number of machines, jobs, and operations within the job shop (Wang & Liao, 2023), in addition to metrics representing the remaining processing time for incomplete jobs and the current load on machines.

However, practical applications often involve a multitude of machines, jobs, and operations, which can be exceptionally extensive, potentially even infinite (Shi, Fan, Xiao, Lin, & Xing, 2020). Directly utilizing these metrics as state features in the input of the neural network may introduce substantial variability, which could lead to suboptimal performance during the training phase of the algorithm. Consequently, the algorithm model may exhibit suboptimal performance when faced with an uncertain environment (Zhu & Zhao, 2022). To address this challenge, we have formulated five state features to represent the production status at a rescheduling point. Confining all state features within the range of  $[0, 1]$ , enabling seamless extension of its applicability to diverse production environments (Friedman & Komogortsev, 2019). This approach enhances the algorithm's optimizing performance and generalization capabilities, allowing it to adapt to a broader spectrum of production scenarios.

#### 3.1. Composition of state space

##### 3.1.1. Definition of state features

Constructing a reasonable state feature for the neural network input is essential, not only to show the instantaneous changes in the system's state, including machine conditions, progress of various jobs, and quantities of completed and pending jobs but also to relate to optimization objective. Improving machine utilization, reducing the waiting time of jobs before processing, and assigning suitable jobs to machines with different processing capacities all contribute to achieving better optimization goals. Before delving into the introduction of the state feature vector in this paper, it is imperative to introduce ourselves with the following parameters.  $TM_k(t)$  is the final completion time of the last operation on  $M_k$  at time  $t$ .  $OJ_i(t)$  is the number of completed operation for  $J_i$  at time  $t$ .  $R_k(t) = \frac{\sum_{j=1}^{OJ_i(t)} P_{i,j,k} X_{i,j,k}}{TM_k(t)}$  is the operation completion rate of  $J_i$  at time  $t$ . Derived from the aforementioned parameters, the representation of the environmental state at time  $t$  is articulated as follows:

1. Average machine utilization  $R_{ave}(t)$ :

$$R_{ave}(t) = \frac{\sum_{k=1}^m R_k(t)}{m} \quad (9)$$

2. Average waiting rate of job's operations  $TRJ_i(t)$ :

$$TRJ_i(t) = \frac{\sum_{j=1}^{OJ_i(t)} \sum_{k=1}^m P_{i,j,k}}{\sum_{j=1}^{OJ_i(t)} \max C_i} \quad (10)$$

3. The rate of job that the machine can process  $M_k P(t)$ :

$$M_k P(t) = \frac{\sum_{k=1}^m \sum_{j=1}^{OJ_i(t)} \sum_{j=OJ_i(t)} O_{i,j} X_{i,j,k}}{(\sum_{i=1}^n n_i - \sum_{i=1}^n OJ_i(t)) * m} \quad (11)$$

4. Estimated proportion of delayed operations  $D_e(t)$ :

If  $T_c \leftarrow \frac{\sum_{k=1}^m TM_k(t)}{m}$  is designated as the average completion time of the last operation on all machines at time  $t$ , the estimated tardiness of  $J_i$  can be viewed as the job for which the average remaining processing time exceeds the slack time from  $T_c$  to  $D_i$ . The estimated tardiness ratio  $D_e(t)$ , representing the proportion of estimated delayed jobs to the total number of jobs, is defined as the ratio of the number of operations for the estimated delayed jobs to the total number of operations  $O_{SUM} = \sum_{i=1}^n n_i$ . The computational methodology is explained in Algorithm 1.

5. The actual delay rate of delayed operations  $D_a(t)$ :

If the completion time of operation  $O_{i,OJ_i(t)}$  is larger than the  $D_i$ , the job  $J_i$  can be defined as a delayed job. Then the  $D_a(t)$  can be described as the ratio of the remaining operations for delay jobs to

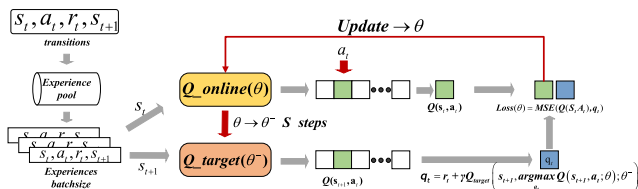


Fig. 1. Training process of DDQN.



---

**Algorithm 1:**  $D_e(t)$ .

---

**Input:**  $T_c, OJ_i(t), D_i$   
**Output:**  $D_e(t)$

```

1  $J_{tard} \leftarrow 0$ 
2 for  $i = 1 : n$  do
3    $T_{left} \leftarrow 0$ 
4   for  $j = OJ_i(t) + 1 : n_i$  do
5      $T_{left} \leftarrow T_{left} + \frac{\sum_{k=1}^m P_{i,j,k} X_{i,j,k}}{|M_{i,j}|}$ 
6     if  $T_c + T_{left} > D_i$  then
7        $J_{tard} \leftarrow J_{tard} + n_i - j + 1$ 
8       break
9     end if
10  end for
11 end for
12  $D_e(t) \leftarrow \frac{J_{tard}}{O_{SUM}}$ 
13 Return  $D_e(t)$ 

```

---



---

**Algorithm 2:**  $D_a(t)$ .

---

**Input:**  $OJ_i(t), D_i$   
**Output:**  $D_a(t)$

```

1  $J_{tard} \leftarrow 0$ 
2 for  $i = 1 : n$  do
3   if  $OJ_i(t) < n_i$  then
4     if  $C_{i,OJ_i(t)} > D_i$  then
5        $J_{tard} \leftarrow J_{tard} + n_i - OJ_i(t)$ 
6     end if
7   end if
8 end for
9  $D_a(t) \leftarrow \frac{J_{tard}}{O_{SUM}}$ 
10 Return  $D_a(t)$ 

```

---

the total number of operations. The detailed calculation of  $D_a(t)$  is illustrated in Algorithm 2.

### 3.1.2. Correlations between states features

In RL-based scheduling methods, the meticulous design of state features constitutes a pivotal factor in ensuring the algorithm's performance. These state features, which direct or indirect expression attributes such as the number of jobs, number of operations, quantity of machines, remaining work hours, remaining number of operations, machine load, and total processing time, form the bedrock of the production state. However, the infinite range of values associated with these features necessitates a subjective delineation and decomposition of the state space, often lacking guidance from objective data. Within the intricate landscape of flexible manufacturing environments, production information undergoes continuous and highly diverse fluctuations. The inclusion of an excessive number of state features, coupled with pronounced inter-feature correlation, poses the risk of inducing overfitting in the model. Conversely, a shortage of state features, accompanied by insufficient inter-feature correlation, may result in underfitting (Shiue, Lee, & Su, 2018). To validate the effectiveness of the proposed state features and mitigate the occurrence of highly correlated state sequences, a comprehensive evaluation of states is conducted by implementing a random policy and collecting data from diverse instances. The experimental results are presented in Fig. 2 below.

The results from the heatmap reveal that the relationships between state features align with expectations—they are neither excessively correlated nor entirely unrelated. This suggests that, during the feature se-

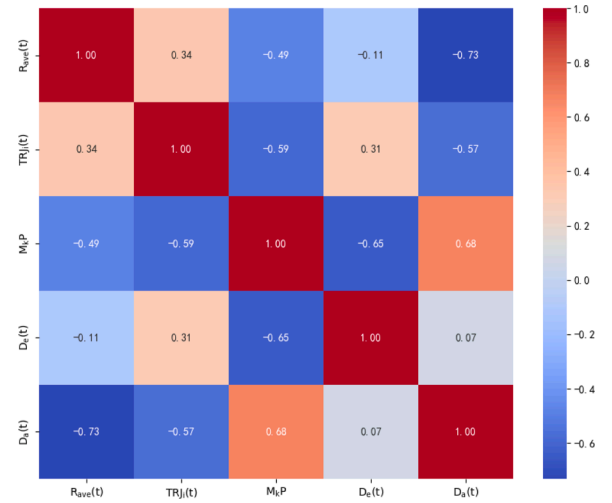


Fig. 2. The heatmap of state features.

lection and design process, we successfully designed the key state features.

### 3.2. Design of action space

Different rules should be utilized corresponding to different production statuses, thus twelve composite dispatch rules are designed as action space of the neural network's output. The actions for machine and operation selection were designed based on the ideas of previous work (Doh, Yu, Kim, Lee, & Nam, 2013; Jeong & Kim, 1998; Priore, Ponte, Puente, & Gómez, 2018). These dispatching rules have been verified to be effective in minimizing tardiness. We redesigned the rules combination based on the characteristics of the DFJSP. Moreover, DFJSP considered in this paper encompasses two sub-problems: job selection and machine assignment. At each decision time  $t$ , the agent is tasked not only with selecting a job but also determining the machine assignment. To address this challenge, the paper defines the set of overdue jobs at time  $t$  denoted as  $Tard_{job}(t)$ , which includes all unfinished jobs with  $D_i < T_c$ . Additionally, the set of all unfinished jobs is defined as  $Proc_{job}(t)$ . Building upon these definitions, twelve composite dispatch rules have been devised, detailed as follows.

1. Dispatch rule 1: In dispatch rule-1, at time  $t$ , the job with the maximum ratio, which is the slack time (from  $T_c$  to the tardiness of each job) to the remaining number of operations. It is assigned to the earliest available machine for processing. If multiple machines are available simultaneously, one is randomly selected for processing. The detailed process for dispatch rule-1 is outlined.

---

**Dispatch Rule-1:**

---

- ```

1  $J_l \leftarrow \arg \min_{i \in Proc_{job}(t)} \frac{D_i - T_c}{n_i - OJ_i(t)}$ 
2  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,OJ_i(t)}))$ 
3  $j \leftarrow OJ_i(t) + 1$ 
4  $Assign / O_{i,j}$  on  $M_k$ 

```
- 

2. Dispatch rule 2: In dispatch rule-2, the job selected for processing at time  $t$  is determined by evaluating the ratio of slack time against the average remaining processing time of each job. The job with the minimum ratio is assigned to the earliest available machine. In situations where multiple machines are simultaneously available, one is randomly chosen for processing. The detailed procedure for dispatch rule-2 is outlined.

**Dispatch Rule-2:**

- 1  $J_i \leftarrow \arg \min_{i \in Proc_{job}(t)} \sum_{j=OJ_i(t)}^{n_i} \frac{(D_i - T_c) |M_{i,j}|}{\sum_{k=1}^m P_{i,j,k} X_{i,j,k}}$
- 2  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j}))$
- 3  $j \leftarrow OJ_i(t) + 1$
- 4 Assign  $O_{i,j}$  on  $M_k$

3. Dispatch rule 3: In dispatch rule-3, at time  $t$ , the procedure involves selecting the job with the highest ratio, which is the estimated tardiness to the remaining number of operations. Then assigning it to the earliest available machine for processing. In cases where multiple machines are available simultaneously, one machine is randomly chosen for processing. The detailed procedure for rule 3 scheduling is outlined.

**Dispatch Rule-3:**

- 1  $J_f \leftarrow \sum_{j=OJ_f(t)}^{n_i} \frac{P_{i,j,k} X_{i,j,k}}{|M_{i,j}|}$
- 2  $J_i \leftarrow \arg \max_{i \in Proc_{job}(t)} [(T_c + J_f - D_i) / (n_i - OJ_i(t))]$
- 3  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j}))$
- 4  $j \leftarrow OJ_i(t) + 1$
- 5 Assign  $O_{i,j}$  on  $M_k$

4. Dispatch rule 4: In dispatch rule-4, the selection process at time  $t$  involves choosing the job with the lowest remaining flexibility and the minimum remaining processing time. Subsequently, machines are prioritized based on the processing time of the next operation for this selected job on each available machine. The assignment is then made to the earliest available machine for processing. In cases where multiple machines are available at the same earliest time, one is randomly chosen for processing. The specific process of dispatch rule-4 is outlined.

**Dispatch Rule-4:**

- 1  $J_f \leftarrow \sum_{j=OJ_f(t)}^{n_i} \frac{P_{i,j,k} X_{i,j,k}}{|M_{i,j}|}$
- 2  $J_i \leftarrow \arg \min_{i \in Proc_{job}(t)} J_f \&\& \min(|M_{i,OJ_i(t)+1}|)$
- 3  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j}))$
- 4  $j \leftarrow OJ_i(t) + 1$
- 5 Assign  $O_{i,j}$  on  $M_k$

5. Dispatch rule 5: In dispatch rule-5, jobs are selected at time  $t$  based on the product of their completion rate and slack time. The job with the smallest product is given priority and assigned to the earliest available machine for processing. If multiple machines are available simultaneously, one is randomly chosen for processing. The detail of dispatch rule-5 is outlined.

**Dispatch Rule-5:**

- 1  $J_i \leftarrow \arg \min_{i \in Proc_{job}(t)} \frac{CJ_i(t)}{n_i} \cdot (D_i - T_c)$
- 2  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j}))$
- 3  $j \leftarrow OJ_i(t) + 1$
- 4 Assign  $O_{i,j}$  on  $M_k$

6. Dispatch rule 6: In dispatch rule-6, at time  $t$ , the job with the maximum estimated tardiness is selected and assigned to the earliest available machine for processing. If multiple machines are available simultaneously, one is randomly chosen for processing. The detailed process of dispatch rule-6 is outlined.

**Dispatch Rule-6:**

- 1  $J_f \leftarrow \sum_{j=OJ_f(t)}^{n_i} \frac{P_{i,j,k} X_{i,j,k}}{|M_{i,j}|}$
- 2  $J_i \leftarrow \arg \max_{i \in Proc_{job}(t)} (T_c + J_f - D_i)$
- 3  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j}))$
- 4  $j \leftarrow OJ_i(t) + 1$
- 5 Assign  $O_{i,j}$  on  $M_k$

7. Dispatch rule 7: In dispatch rule-7, at time  $t$ , the job selected for processing is the one with the earliest available processing time,

allocated to the earliest available machine. In scenarios where multiple machines are available simultaneously, the machine with the minimum load is chosen for processing. The detailed process of dispatch rule-7 is outlined.

**Dispatch Rule-7:**

- 1  $J_i \leftarrow \arg \max_{i \in Proc_{job}(t)} (\min(CJ_i(t)))$
- 2  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j})) \&\& \min(R_k)$
- 3  $j \leftarrow OJ_i(t) + 1$
- 4 Assign  $O_{i,j}$  on  $M_k$

8. Dispatch rule 8: In dispatch rule-8, at time  $t$ , the job with the shortest processing time for the pending operation is selected and assigned to the earliest available machine for processing. When multiple machines are simultaneously available, the machine with the minimum load is chosen for the task. The detailed procedure of dispatch rule-8 is outlined.

**Dispatch Rule-8:**

- 1  $J_i \leftarrow \arg \min_{i \in Proc_{job}(t)} (P_{i,OJ_i(t)+1,k})$
- 2  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j})) \&\& \min(R_k)$
- 3  $j \leftarrow OJ_i(t) + 1$
- 4 Assign  $O_{i,j}$  on  $M_k$

9. Dispatch rule 9: In dispatch rule-9, at time  $t$ , the job with the longest remaining processing time is selected and assigned to the earliest available machine for processing. When multiple machines are available simultaneously, the machine with the lowest load is chosen for processing. Dispatch rule 9 has provided a detailed exposition of the intricate steps.

**Dispatch Rule-9:**

- 1  $J_i \leftarrow \arg \max_{i \in Proc_{job}(t)} (P_{i,OJ_i(t)+1,k})$
- 2  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j})) \&\& \min(R_k)$
- 3  $j \leftarrow OJ_i(t) + 1$
- 4 Assign  $O_{i,j}$  on  $M_k$

10. Dispatch rule 10: In dispatch rule-10, the job with the shortest remaining processing time is selected at time  $t$ , excluding the processing time for the current pending operation. Subsequently, this job is assigned to the earliest available machine for processing. If multiple machines are available simultaneously, the one with the lowest machine load is chosen for processing. The detailed procedure for dispatch rule-10 is explained as follows.

**Dispatch Rule-10:**

- 1  $J_{f+1} \leftarrow \sum_{j=OJ_f(t)+1}^{n_i} \frac{P_{i,j,k} X_{i,j,k}}{\sum_{k=1}^m P_{i,j,k}}$
- 2  $J_i \leftarrow \arg \min_{i \in Proc_{job}(t)} (J_{f+1})$
- 3  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j})) \&\& \min(R_k)$
- 4  $j \leftarrow OJ_i(t) + 1$
- 5 Assign  $O_{i,j}$  on  $M_k$

11. Dispatch rule 11: In dispatch rule-11, at time  $t$ , the job with the lowest completion rate is selected and assigned to the earliest available machine for processing. When multiple machines are available simultaneously, the machine with the minimum load is chosen for processing. The detailed process of dispatch rule-11 is outlined.

**Dispatch Rule-11:**

- 1  $J_i \leftarrow \arg \min_{i \in Proc_{job}(t)} \frac{OJ_i(t)}{n_i}$
- 2  $M_k \leftarrow \arg \min_{k \in M_{i,j+1}} (max(TM_k(t), C_{i,j})) \&\& \min(R_k)$
- 3  $j \leftarrow OJ_i(t) + 1$
- 4 Assign  $O_{i,j}$  on  $M_k$

12. Dispatch rule 12: In dispatch rule-12, the selection process at time  $t$  begins by choosing the earliest available machine, followed by selecting the job that can be processed the earliest on that machine.

If the chosen machine is incapable of processing all remaining jobs, prioritization is based on the earliest processing times of machines. Moreover, If multiple jobs can be processed at the earliest, select the jobs with the lowest completion rate among them. The detailed process of dispatch rule-12 is outlined. It is noteworthy that in many existing studies, composite dispatch rules often prioritize job selection before machine assignment, overlooking machine flexibility. In environments where not all machines possess universal flexibility, late-stage scheduling may result in certain operations being constrained to specific machines. This can lead to subsequent operations being blocked due to machine preemption, thereby constraining the performance of scheduling algorithms.

---

**Dispatch Rule-12:**


---

- 1  $M_k \leftarrow \arg \min (TM_k(t))$
  - 2  $J_i \leftarrow \arg \min_{j \in Proc_{jod}(t)} (C_{i,OJ_j(t)})$
  - 3  $j \leftarrow OJ_i(t) + 1$
  - 4 Assign  $O_{i,j}$  on  $M_k$
- 

### 3.3. Reward function

For the state-action pair  $(s_t, a_t)$  at time  $t$ , the reward  $r_t$  is determined by changes in five state features during the system's transition from state  $s_t$  to  $s_{t+1}$ . These features include two directly associated with the optimization objective and three indirectly linked to secondary objectives. The primary goal of the reward function is to minimize the overall tardiness of jobs. However, in the initial phases of job shop scheduling, where jobs have not encountered tardiness, exclusive emphasis on the two tardiness-related features could result in highly coupled states. This interdependence hinders the agent's discernment of distinct production scenarios. Consequently, the reward function incorporates three secondary features. The detailed procedure for computing  $r_t$  is elucidated in Algorithm 3.

---

**Algorithm 3: Reward.**


---

- 1 Delay  $\leftarrow (\frac{D_e(t)}{D_e(t+1)})D_e(t) + (\frac{D_a(t)}{D_a(t+1)})D_a(t)$
  - 2 Affect  $\leftarrow (\frac{R_{ave}(t+1)}{R_{ave}(t)})R_{ave}(t) + (\frac{TRJ_i(t+1)}{TRJ_i(t)})TRJ_i(t) + (\frac{M_k P(t+1)}{M_k P(t)})M_k P(t)$
  - 3 reward  $\leftarrow Affect - Delay$
  - 4 return reward
- 

In accordance with Algorithm 3, the reward comprises two components: Delay and Affect. Delay encompasses two state values directly associated with the optimization objective, while effect is composed of three states indirectly related to the optimization objective. Dense reward functions exhibit higher efficiency during early exploration phases, yet designing a globally optimal dense reward function proves challenging and is susceptible to noise interference. Conversely, sparse reward functions demonstrate lower efficiency during early exploration; however, if valuable samples are provided, agents guided by sparse rewards theoretically exhibit greater consistency with the task and stronger robustness against noise. To address the limitations of both rewards, this study also incorporates special sparse rewards. The detailed implementation of this approach will be discussed subsequently in conjunction with the improvements introduced in the experience pool structure.

#### 3.3.1. The application of prioritized experience replay

The formulation of prioritized experience replay is based on a nuanced consideration of the importance of experiences within DRL algorithms. By discerning the significance of each experience, the training process achieves heightened efficiency and precision. Diverging from conventional experience replay methodologies, prioritized experience

replay strategically samples experiences based on their temporal difference (TD) errors, ensuring that pivotal experiences are given a higher frequency in training iterations. The training regimen employs a Priority Buffer data structure to archive experiences, where each experience is endowed with a specific priority. Experiences with elevated priorities undergo more frequent sampling. When recalibrating priorities, due diligence is given to the TD error of each experience to aptly reflect its consequential.

Within the Q-network, the TD error manifests as the variance between the target Q-value of the target network and the estimated Q-value of the online network. Prioritized Replay DQN systematically computes the priority of each sample based on its TD error. The loss function, cognizant of sample priorities, is articulated as follows:

$$L_i(\theta_i) = \mathbb{E}[(q_i)^{DoubleDQN} - Q(s, a; \theta_i)]^2 \quad (12)$$

#### 3.3.2. The improvement of experience pool structure

Integrating prioritized experience replay enhances the algorithm's ability to selectively, utilize significant experiences within a limited number of iterations. However, when the experience pool expands considerably, the algorithm may struggle to harness the entirety of experiences within a finite iteration. This makes the problem of potential entrapment in local optima. Conversely, in scenarios characterized by a small experience pool and a large solution space, valuable experiences might quickly pop from the pool, rendering them ineffectual. To counterbalance these challenges, this paper advocates for an improved experience pool structure, illustrated in Fig. 3(b), to augment the algorithm's short-time intelligent seeking capabilities.

In the domain of DRL, the challenge of local optima frequently arises due to the vast expanse of the state space, making the pursuit of a global optimum profoundly intricate. In instances where the dimension of the state space is substantial, neural networks may encounter challenges in adequately representing and assimilating all facets and nuances of the state space. This constrains their exploration prowess for global optima. The reward function, serving as a conduit for mapping between the state space and action space, assumes a pivotal role. A well-crafted reward function facilitates the agent in discerning action values under diverse states.

Hence, for intricate problems, the repercussions of potential intricacies in the reward function on RL algorithms assume paramount significance. In the realm of RL, the Bellman expectation equation emerges as a pivotal formula delineating the state value or state-action value within MDP. It elucidates the interrelation between the current state value (or state-action value) and forthcoming state values. For the state value function, the Bellman expectation equation manifests as follows:

$$V(s) = \max \left( R(s, \alpha) + \gamma \sum_a P(s_{t+1}|s, a) V(s_{t+1}) \right) \quad (13)$$

For the state action value function, the Bellman expectation equation is as follows:

$$Q(s, \alpha) = R(s, a) + \gamma \sum_{s_{t+1}} P(s_{t+1}|s, a) \max Q(s_{t+1}, a_{t+1}) \quad (14)$$

In the DDQN, the agent acquires knowledge of the state-action value function  $Q(s, a)$  through

$$q_t^{DDQN} = r_t + \gamma \hat{Q} \left( s_{t+1}, \arg \max_{a_t} Q(s_{t+1}, a_t; \theta); \theta^- \right) \quad (15)$$

$r_t$  is calculated by Algorithm 3. This learning process is driven by the optimization of the scheduling objective, where the agent approximates and updates the Bellman optimal equation

$$V^*(s) = \max_a \sum_{s_{t+1}, r} p(s_{t+1}, r | s, a) [r + \gamma V^*(s_{t+1})] \quad (16)$$

The neural network plays a pivotal role in approximating and updating the state-action values in the Bellman expectation equation. By iteratively updating network parameters  $\theta$  with experiential data, the model

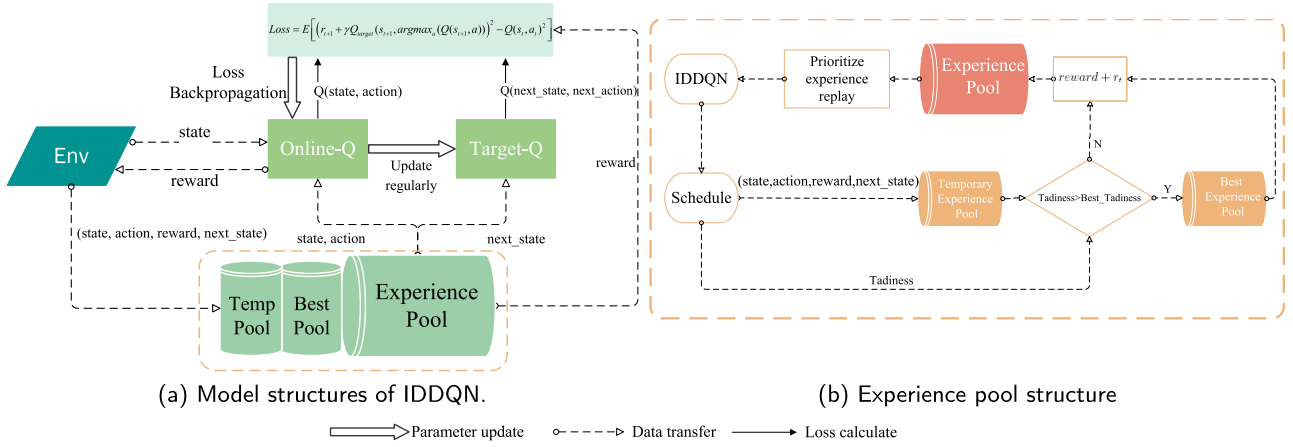


Fig. 3. The framework of IDDQN.

gradually converges to the optimal state-action value function, achieving an optimal scheduling solution.

Through the equation, it is apparent that  $Q(s, a)$  at the current state relies on the immediate reward  $r_t$  and the maximum  $Q(s, a)$  at the subsequent state. However, as the optimization objective in this paper is total tardiness, discernible only after scheduling completion, immediate rewards for each action inadequately capture the goodness or badness of the optimization objective. Therefore, deviating from conventional DDQN iterative training, this paper redefines  $q_t$  as follows.

$$q_t^{IDDQN} = r_t + \gamma \hat{Q}(s_{t+1}, \arg\max_{a_t} Q(s_{t+1}, a_t; \theta); \theta^-) + r_T \quad (17)$$

For experiences in the temporary memory pool,  $r_T = (Best\_Tardiness - Tardiness)/O_{sum}$ . The others in the optimal memory pool  $r_T = (Tardiness - Best\_Tardiness)/O_{sum}$ . The reward  $r_T$  serves as a distinctive sparse reward for each action, effectively guiding the agent to swiftly align its actions with the optimization objectives.

### 3.3.3. Neural networks

The neural architecture employed in this study constitutes a profound neural network, comprising five fully connected layers, encompassing one input layer, one output layer, and three concealed layers. The number of nodes within the input and output layers aligns with the count of state features and available actions, respectively. Within this paper, the "ReLU" activation function is deployed to activate the input and hidden layers, while the "purelin" activation function is employed for the output layer.

The Dueling DQN aspires to augment algorithmic efficacy through the refinement of the neural network structure. This enhancement is realized by introducing two subnetworks, each specifically tasked with learning the value and advantage functions. The primary goal of this network structure is to enable the model's output to concurrently factor in both the value and advantage functions. This integration results in the generation of more precise and comprehensive state-action values. The merit of this network architecture lies in its capacity to more adeptly comprehend and represent information about the state value function, thereby amplifying the model's performance and proficiency in addressing challenges within RL. In this study, the Dueling framework proposed by Wang et al. (2016) is integrated into the DDQN framework. The detailed network structure is delineated in Fig. 4.

### 3.4. Training process detail of IDDQN

In this context, the decision point  $t$  is defined to occur either upon a machine breakdown or upon the completion of an operation. The termination signal  $T$  signifies the conclusion of the scheduling process, indicating the completion of all operations.  $L$  represents the count of

Table 2

The parameters of IDDQN.

| Parameter                                | Value               |
|------------------------------------------|---------------------|
| Learning rate                            | 0.01                |
| Number of training iterations            | $2 \times 10^3$     |
| The capacity of experience pool          | $10 \times O_{sum}$ |
| Sample batch size                        | $O_{sum}$           |
| Coefficients in greedy strategy $\theta$ | 0.99 to 0.001       |
| Update steps                             | $O_{sum}$           |
| Discount factor $\gamma$                 | 0.9                 |
| Prioritize experience replay alpha       | 0.6                 |
| Prioritize experience replay beta0       | 0.4                 |

training iterations, while  $S$  is the synchronization time of the two neural networks. The training methodology, rooted in the IDDQN, is meticulously outlined in Algorithm 4.

## 4. Computational experiments and results

Parameter calibration, ablation, and comparison experiments are designed to test the performance of IDDQN. IDDQN is coded in Python 3.8 with CUDA 11.3 and PyTorch. The running environment is on 12th Gen Intel®Core™i5-12500H @ 2.50 GHz, 40.00 GB RAM, RTX3060, Win11 64.

### 4.1. Experiment preparation

The proposed IDDQN is trained in a simulated flexible job shop environment, the simulation data are generated based on the number of jobs  $n \in \{5, 10, 15, 20, 30, 50, 100\}$ , the number of machines  $m \in \{5, 10, 15, 20, 30\}$ . We evaluate the proposed algorithm on the standard benchmark. Some hyperparameters during training are shown in Table 2. These parameters are obtained by hyperparameter optimization using grid search.

The instances utilized in the experiments are outlined in Table 3. All benchmarks are static, assuming the readiness of all jobs at initial-ization time. To ensure the manifestation of tardiness for algorithmic performance verification and to limit excessive flexibility in delivery times, the tardiness for each job is set to 80% of its average processing time, i.e.,  $D_i = \sum_{j=0}^{n_i} \frac{\sum_{k=1}^m P_{i,j,k} X_{i,j,k}}{|M_{i,j}|} \times 80\%$ . It is noteworthy that in the production environment of the natural world, tardiness is typically not as urgent as defined in this paper.

### 4.2. Ablation study

To evaluate the effectiveness of the improvement part in IDDQN, We also compared IDDQN with the unimproved DDQN under the same



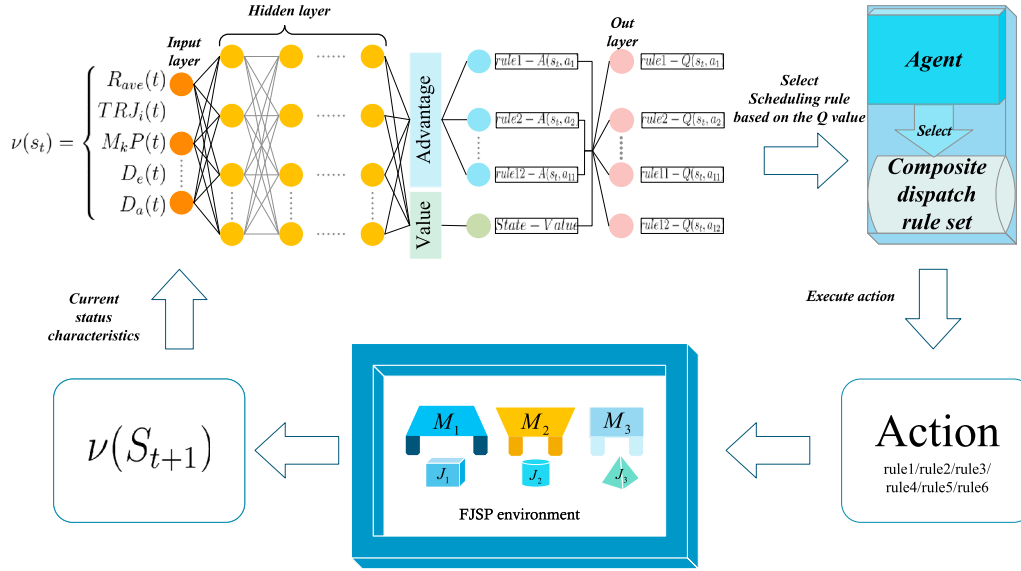


Fig. 4. The structure and applications of neural networks.

**Algorithm 4:** The training process detail of IDDQN.

```

1 Initialize replay memory Pool, Pooltemp, Poolbest
2 Initialize network Q and  $\hat{Q}$  with random weights
3 Initialize Dbest = 0
4 for episode = 1 : L do
5   Generate the initial state s1 with feature vector v(1) =
   {Rave(1), TRJi(1), MkP(1), De(1), Da(1)} = {0, 0, 0, 0, 0}
6   for t = 1 : T do
7     Select at = arg maxa Q(vt, a; θ) with Probability ε
8     Execute action at, observe the next state st+1
9     Calculate the immediate reward rt by Algorithm 3
10    Obtain the new feature vector vt+1 = {Rave(t +
    1), TRJi(t + 1), MkP(t + 1), De(t + 1), Da(t + 1)}
11    Store transition (vt, at, rt, vt+1) in Pooltemp
12  end for
13  Obtain the D
14  if D < Dbest then
15    set Dbest = D
16    Store transition (vt, at, rt, vt+1) in Poolbest
17    set rj = rt + rT from Poolbest
18    Store transition (vt, at, rj, vt+1) in Pool
19  else
20    set rj = rt + rT from Pooltemp
21    Store transition (vt, at, rj, vt+1) in Pool
22  end if
23  Store Pooltemp and Poolbest in Pool
24  Sample Osum minibatch of transitions (vj, aj, rj, vj+1)
  from Pool
25  Set yj =
  { rj if episode terminates at step j + 1
  { rj + γQ̂(vj, argmaxaQ(vj, aj; θ); θ-) otherwise
26  Do a gradient descent step with loss (yj - Q(vj, aj; θ))2
27  Replace Q = Q̂ every S steps
28 end for

```

**Table 3**

Benchmarks of testing and training.

| Benchmarks                         | Source                          |
|------------------------------------|---------------------------------|
| abz5, abz6, abz7, abz8, abz9       | Adams, Balas, and Zawack (1988) |
| la30, la31, la32, la33, la34, la35 | Lawrence (1984)                 |
| mt06, mt10, mt20                   | Fisher (1963)                   |
| Mk01, Mk05, Mk10                   | Brandimarte (1993)              |

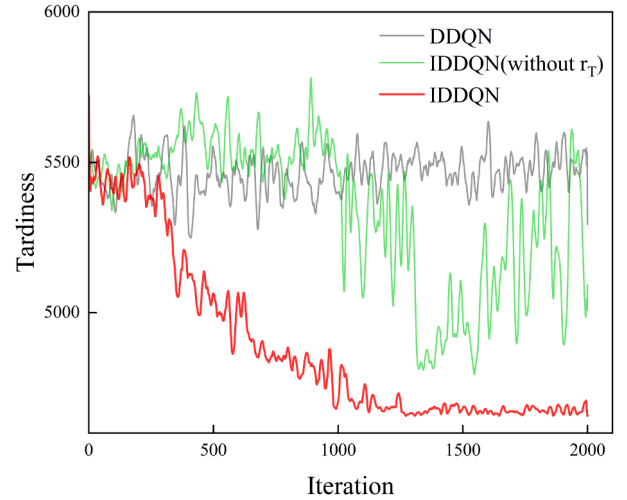


Fig. 5. Tardiness of different DRL algorithms.

conditions. DDQN will lack a distinctive experience pool architecture and  $r_T$ , using the normal iterative approach. The results of the training under identical conditions are presented in Fig. 5. The figure illustrates the total tardiness outcomes achieved by IDDQN within the initial 2000 training steps. The total tardiness curve demonstrates a gradual decline with increasing training steps, affirming that the proposed IDDQN adeptly learns to dynamically formulate dispatch rules across diverse states.

The results are presented in Table 4. With the increase of data scale in Table 4, the disparity between the maximum and minimum total tardiness becomes more pronounced. This is attributed to the difference number of operations pending processing after machine breakdown, thereby

**Table 4**  
IDDQN compare with DDQN.

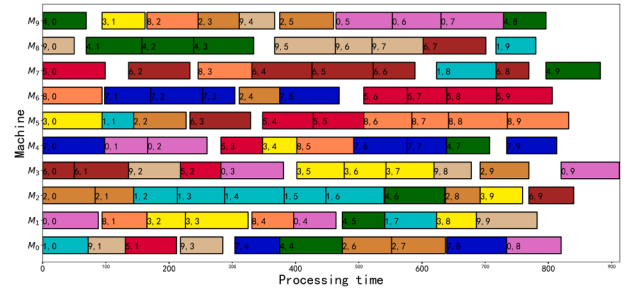
| Data | IDDQN |         |       | DDQN  |         |       |
|------|-------|---------|-------|-------|---------|-------|
|      | Max   | Average | Min   | Max   | Average | Min   |
| abz5 | 3158  | 3125    | 3092  | 4468  | 4114    | 3575  |
| abz6 | 2296  | 2211    | 2189  | 3632  | 3368    | 3025  |
| abz7 | 4539  | 4411    | 4341  | 7731  | 7113    | 6617  |
| abz8 | 4545  | 4495    | 4466  | 8678  | 7269    | 6441  |
| abz9 | 4457  | 4397    | 4373  | 7664  | 7206    | 6839  |
| la30 | 10726 | 10584   | 10495 | 13714 | 13404   | 12959 |
| la31 | 29392 | 29018   | 28462 | 38027 | 35528   | 32461 |
| la32 | 35503 | 34457   | 33634 | 42131 | 40319   | 38526 |
| la33 | 29722 | 29151   | 28471 | 34986 | 33290   | 31735 |
| la34 | 32228 | 31536   | 31002 | 38836 | 36792   | 35376 |
| la35 | 35377 | 34308   | 33394 | 44931 | 41785   | 40022 |
| mt06 | 89    | 79      | 74    | 156   | 122     | 74    |
| mt10 | 2049  | 2028    | 2012  | 3317  | 2875    | 2671  |
| mt20 | 13591 | 13359   | 13037 | 17858 | 16596   | 15403 |
| Mk01 | 146   | 137     | 130   | 239   | 210     | 177   |
| Mk05 | 1773  | 1689    | 1615  | 2579  | 2371    | 2241  |
| Mk10 | 1591  | 1581    | 1557  | 2812  | 2363    | 1988  |

amplifying the uncertainty of the problem. This result can be attributed to the fact that the influence of uncertainty on tardiness increases as the sizes of state and action spaces grow.

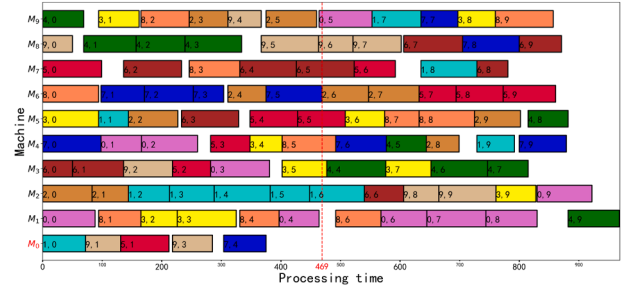
From the result of Wilcoxon experiment in Table 5, it is evident that the algorithm exhibits significant differences after improvement. DDQN achieves inferior optimization results compared to IDDQN under identical conditions. This is due to the longer training cycles required by DDQN, stemming from lower learning efficiency and erroneous learning experiences.

In the early episode of training, the acquisition of a significant amount of irrelevant experiences may lead the agent into an optimization dilemma prematurely. This situation can render subsequent relevant and correct experiences ineffective. Due to the specific optimization objectives chosen for job shop scheduling, conventional DDQN algorithms struggle to establish a direct connection between the reward function and the optimization objectives. The proposed IDDQN algorithm, however, strategically executes iterations after each scheduling round, thereby mitigating the potential convergence to local optima that can occur with an abundance of frequent iterations. It utilizes a unique experience pool structure and an efficient reward function to combine sparse and dense rewards, achieving comprehensive surpass over traditional DDQN. It is very important to quickly learn the best scheduling plan for the agent when the uncertain situation cannot be handled and learn to deal with it in the future. The average training time of IDDQN and DDQN on different scale instances is shown in Table 6 and the better value is marked in bold. Since the number of machines affects the scheduling objectives but has minimal impact on the training speed, we have not listed the all. It can be seen that IDDQN can still get better scheduling results by significantly reducing training time. This means our algorithm can quickly retrain the model, even if it encounters an unsolvable problem. It can use technologies like fusion models or stacked models to make existing models more robust. In a real production environment, the scale of the scheduling problem is often uncertain, and a rescheduling time that is too long will lead to an inability to catch up with the real-time production status.

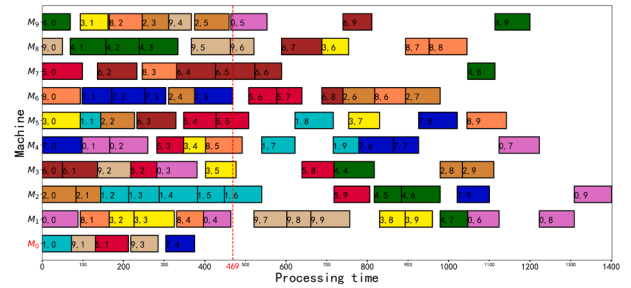
Fig. 6 illustrates Gantt charts depicting scheduling solutions derived through the IDDQN algorithm before and after the occurrence of dynamic disturbance events, respectively. The horizontal axis represents time, while the vertical axis corresponds to machines. Each rectangular bar in the chart symbolizes the duration of processing for a specific operation, with numerical annotations on the bars (e.g., (6, 3)) indicating the third operation of the sixth job. Red dashed lines delineate the time of machine breakdown. It is evident from the charts that in the event of a failure in machine  $M_0$ , the IDDQN algorithm adeptly employs a partial rescheduling approach to formulate a new production plan, thereby mitigating the impact of machine breakdown on the production schedule.



(a) Machine without breakdown.



(b) IDDQN solves machine breakdown.



(c) DDQN solves machine breakdown.

Fig. 6. Gantt chart of abz5.

IDDQN is capable of making more rational decisions at each scheduling selection point than DDQN.

#### 4.3. IDDQN compared with DDPG

To further validate the effectiveness of the proposed algorithm, we compared IDDQN with DDPG (Gui et al., 2023) on the same data. The results are shown in Fig. 7. The results demonstrate that IDDQN outperforms DDPG in most cases. This suggests the proposed algorithm effectively addresses the scheduling challenge in a flexible job shop. Additionally, the algorithm exhibits minimal solution errors, making it well-suited for dynamic scenarios that require rapid responses. Given the dynamic interference, we clearly need a faster and more stable algorithm. IDDQN is more appropriate in this context since we have delicately redesigned the algorithm.

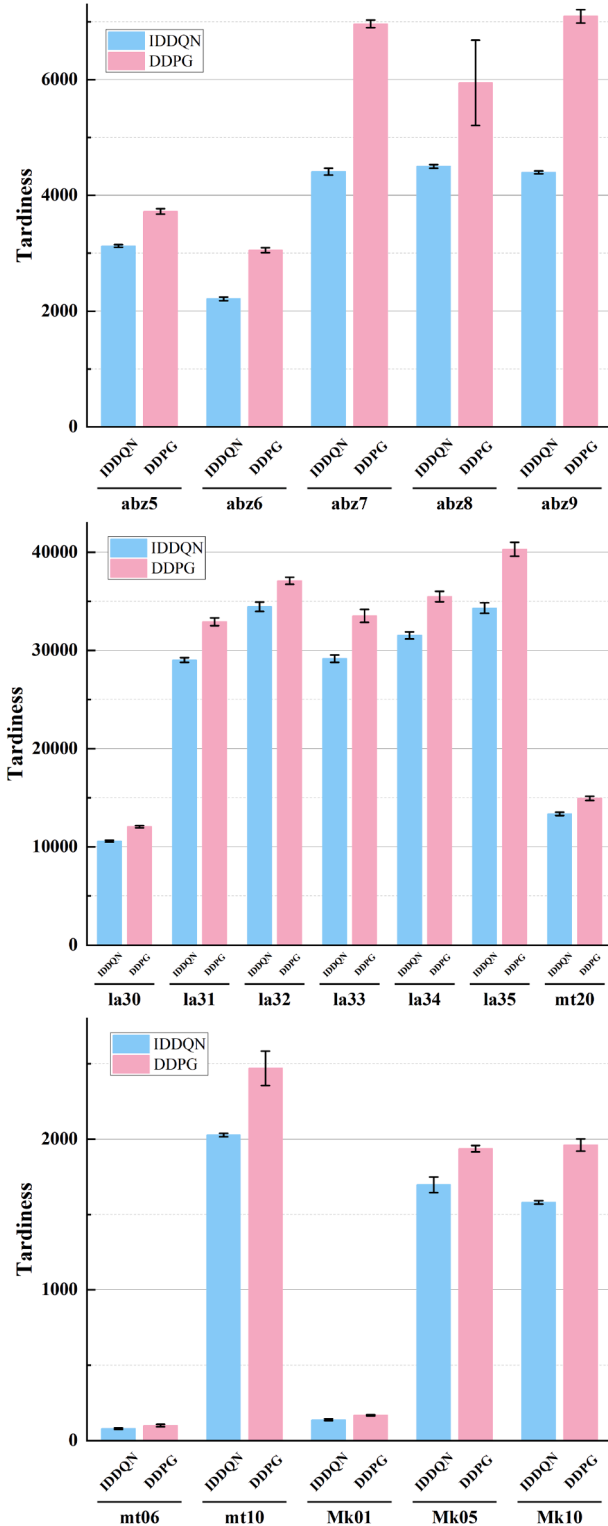
#### 4.4. IDDQN compared with dispatch rules

In IDDQN, twelve dispatching rules assumed the role of actions by the agent. Subsequently, the agent after training makes the decision sequences generated by these rules, yielding a scheduling outcome that performs better than the individual rule. Therefore, the scheduling results of the proposed IDDQN on all instances are first compared with those of dispatch rules.

Benchmark datasets are utilized to assess the generality and performance of IDDQN. For each instance, IDDQN and each composite rule un-

**Table 5**  
The result of Wilcoxon.

| Algorithm     | Max   |       |                  | Average |       |                  | Min   |       |                  |
|---------------|-------|-------|------------------|---------|-------|------------------|-------|-------|------------------|
|               | $R^+$ | $R^-$ | $p\text{-value}$ | $R^+$   | $R^-$ | $p\text{-value}$ | $R^+$ | $R^-$ | $p\text{-value}$ |
| IDDQN vs DDQN | 0     | 17    | 0.001            | 0       | 17    | 0.001            | 0     | 16    | 0.001            |



**Fig. 7.** IDDQN compare with DDPG.

**Table 6**

Training time for IDDQN and DDQN.

| Data    | IDDQN/s        | DDQN/s   |
|---------|----------------|----------|
| n5m5    | <b>65.43</b>   | 2955.99  |
| n10m5   | <b>244.83</b>  | 11060.91 |
| n15m10  | <b>691.59</b>  | 28987.63 |
| n20m15  | <b>1046.09</b> | 33045.01 |
| n30m20  | <b>1366.22</b> | 47586.57 |
| n50m20  | <b>2477.73</b> | 86104.69 |
| n100m30 | <b>5723.55</b> | 19896.69 |

dergo independent repetitions 10 times to ascertain performance consistency across various data. Testing is conducted on a subset of instances derived from the OR library. The original dataset lacked provision for machine breakdown disturbances. Hence, random times following the Weibull distribution are introduced to simulate machine breakdown disturbances, which are generated during the scheduling process and have been described in detail by the formulation Eq. (8).

Each test set underwent ten independent repetitions, and the averages were computed, disregarding decimal values. The mean values of total tardiness obtained by each method are provided in Table 7 with the best results highlighted in bold font.

In all benchmarks of Table 7, IDDQN is significantly better than the single dispatch rule. This implies that the proposed algorithm has learned to select appropriate dispatch rules across diverse scheduling states.

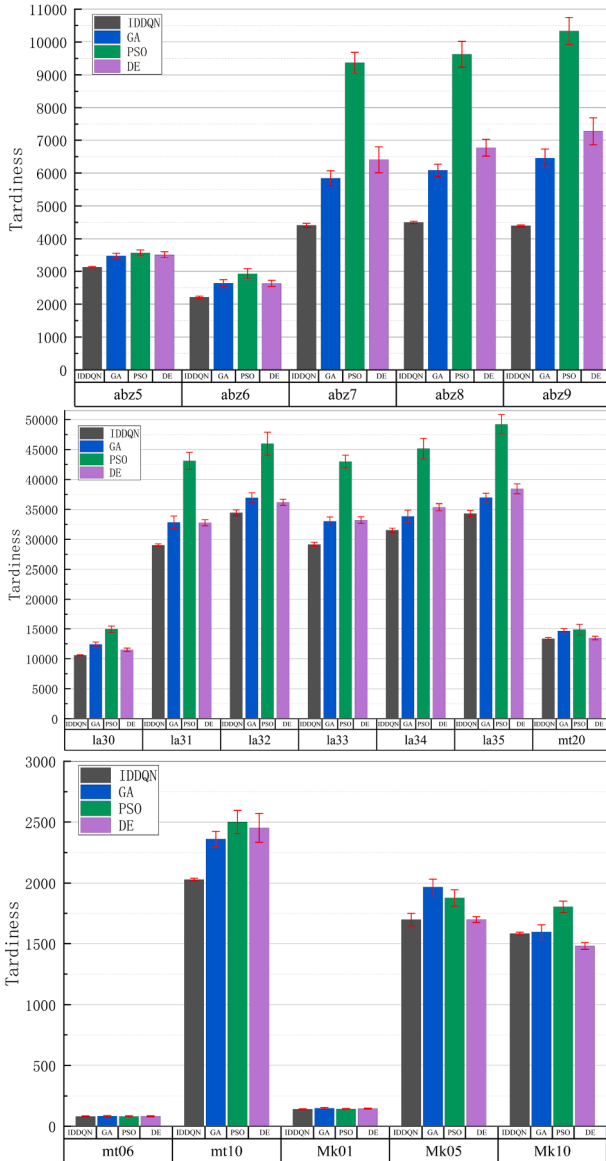
#### 4.5. IDDQN compared with meta-heuristic algorithms

Presently, there is a paucity of DRL job shop research comparing optimization algorithms. Consequently, this study compared IDDQN against PSO (Nouiri, Bekrar, Jemai, Niar, & Ammari, 2018), GA (Fan, Zhang, Liu, Shen, & Gao, 2022) and DE (Ma, He, Wang, Han, & Shi, 2018) under identical data for comparison purposes in Fig. 8 and Table 8. The parameter setting of the meta-heuristic algorithm is to select a medium-sized case, use the orthogonal experimental method, run it 10 times, and take the optimal average value to determine the optimal parameters. The parameters of GA (*population size*,  $P_c$ ,  $P_m$ ) are set to (100, 0.85, 0.3), PSO (*population size*,  $w$ ,  $c_1$ ,  $c_2$ ) are set to (150, 0.7, 0.5, 0.2), DE (*population size*,  $F$ ,  $C_r$ ) are set to (250, 0.5, 0.25).

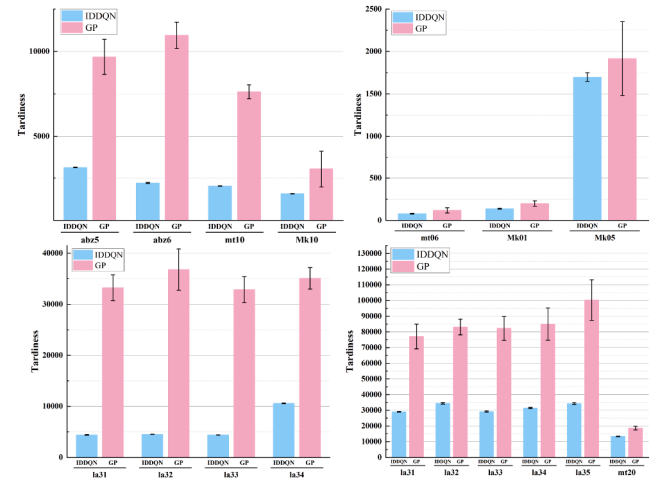
It can be seen that IDDQN has better performance than GA and PSO in most instances. This indicates that the proposed algorithm can effectively solve the scheduling problem in the flexible job shop environment. The algorithm with smaller solution errors is more suitable for dynamic scenarios that require fast response. The stability of our proposed algorithm is superior to that of the meta-heuristic algorithm across multiple benchmarks. Have to be aware of this, Meta-heuristic algorithms lack memory, necessitating repeated solving even when confronted with the same problem. In contrast, the proposed algorithm can learn from previous experiences, thereby enhancing scheduling efficiency. We maintain consistency in the solve time of the optimization algorithm throughout our study, the solve time was set to be positively correlated with the problem size and was set to one-fourth of the product of the number of jobs and the number of machines. Therefore, it is listed only once in the table to avoid duplication. The IDDQN model, which has been trained, takes significantly less time to find the solution compared to the optimization algorithm while maintaining similar solution quality.

**Table 7**  
IDDQN compare with dispatch rules.

| Data | IDDQN | Rule1 | Rule2 | Rule3 | Rule4 | Rule5 | Rule6 | Rule7 | Rule8 | Rule9 | Rule10 | Rule11 | Rule12 | Random |
|------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|--------|--------|--------|--------|
| abz5 | 3125  | 4191  | 4221  | 4588  | 4586  | 4191  | 5057  | 3180  | 4698  | 4993  | 5574   | 3760   | 3180   | 4138   |
| abz6 | 2211  | 4208  | 4296  | 3501  | 3847  | 4208  | 4231  | 2330  | 4392  | 4442  | 4322   | 3022   | 2352   | 3377   |
| abz7 | 4411  | 8778  | 8113  | 8851  | 8990  | 8778  | 8305  | 4445  | 9686  | 8926  | 13261  | 4922   | 4460   | 7690   |
| abz8 | 4561  | 8818  | 9191  | 8788  | 8045  | 8868  | 10395 | 4555  | 8851  | 10418 | 11875  | 5165   | 4607   | 7883   |
| abz9 | 4397  | 7864  | 8583  | 8065  | 8251  | 7814  | 8086  | 4517  | 9282  | 12000 | 10744  | 5564   | 4521   | 7200   |
| la30 | 10584 | 13101 | 13715 | 14746 | 14279 | 13285 | 14261 | 11361 | 17137 | 21776 | 16419  | 15604  | 11240  | 13037  |
| la31 | 29018 | 35421 | 37403 | 38208 | 41165 | 36304 | 34705 | 32751 | 43593 | 50821 | 43954  | 37012  | 32839  | 35294  |
| la32 | 34457 | 43514 | 41212 | 44594 | 43055 | 43063 | 42591 | 36408 | 47506 | 53897 | 45767  | 40455  | 36901  | 39925  |
| la33 | 29151 | 38120 | 39305 | 38587 | 39753 | 38849 | 38073 | 33317 | 42103 | 55702 | 44443  | 36589  | 32953  | 34254  |
| la34 | 31536 | 43334 | 39651 | 42092 | 41358 | 45328 | 42149 | 34357 | 47818 | 55414 | 44997  | 36854  | 34646  | 38457  |
| la35 | 34308 | 47701 | 48003 | 51165 | 49873 | 56278 | 46282 | 37173 | 47137 | 55310 | 53671  | 40422  | 37392  | 41596  |
| mt06 | 79    | 158   | 121   | 189   | 161   | 138   | 155   | 97    | 125   | 142   | 224    | 120    | 124    | 140    |
| mt10 | 2028  | 3219  | 3028  | 3219  | 2661  | 3319  | 3550  | 2128  | 3644  | 3919  | 4148   | 3068   | 2163   | 2924   |
| mt20 | 13359 | 17385 | 17288 | 20593 | 20799 | 20095 | 16342 | 18151 | 18832 | 19947 | 19042  | 19322  | 18151  | 15589  |
| Mk01 | 137   | 227   | 221   | 221   | 219   | 215   | 235   | 178   | 222   | 263   | 229    | 221    | 194    | 217    |
| Mk05 | 1689  | 2360  | 2529  | 2159  | 2632  | 2356  | 2357  | 2115  | 2599  | 3423  | 2735   | 2562   | 2115   | 2070   |
| Mk10 | 1581  | 2633  | 2552  | 2481  | 2672  | 2582  | 2622  | 1801  | 3514  | 2778  | 2758   | 2258   | 1722   | 2394   |



**Fig. 8.** IDDQN compare with meta-heuristic algorithms.



**Fig. 9.** IDDQN compare with GP.

#### 4.6. IDDQN compared with hyperheuristic algorithm

To further compare the comprehensiveness of the IDDQN, we also compared it with the GP algorithm. In our experiment, the terminals are set as features that indicate characteristics related to machines, operations, and jobs. The function set is  $\{+, -, *, /, \max, \min\}$ , following the setting in [Park, Mei, Nguyen, Chen, and Zhang \(2018\)](#). The “/” operator is a protected division, returning one if divided by zero. The hyperparameters of GP (*populationsize*,  $P_c$ ,  $P_m$ , *Gen*) are set to (1024, 0.8, 0.1, 51). The results are shown in [Table 9](#). The GP algorithm performs excellently on a small test set, but its performance degrades when the scale increases due to its lack of generalization. The IDDQN algorithm, as seen from the [Fig. 9](#), has a strong generalization ability and can solve large-scale problems.

## 5. Conclusion and future work

In this study, we proposed a novel framework based on DRL for a dynamic flexible job shop. Our innovative framework incorporates Dueling DQN and prioritized experience replay. To address the intricacies of the scheduling environment, we leverage artificial feature engineering, transforming the scheduling state into a low-dimensional feature space at each temporal step. This abstraction encapsulates the complexity of the production milieu into a more manageable state space, enhancing both stability and algorithmic performance. The action space within our architectural design integrates dispatch rules, making it readily executable in complex operational settings. The refined structure of the ex-



**Table 8**  
IDDQN compare with meta-heuristic algorithms.

| Data | IDDQN |         |       |        |         | GA    |         |       |         |       | PSO     |       |         |       |         | DE    |        |       |  |  | Time(s) <sup>a</sup> |
|------|-------|---------|-------|--------|---------|-------|---------|-------|---------|-------|---------|-------|---------|-------|---------|-------|--------|-------|--|--|----------------------|
|      | Max   | Average | Min   | SD     | Time(s) | Max   | Average | Min   | SD      | Max   | Average | Min   | SD      | Max   | Average | Min   | SD     |       |  |  |                      |
| abz5 | 3158  | 3125    | 3092  | 22.08  | 1.88    | 3632  | 3480    | 3371  | 80.95   | 3761  | 3570    | 3444  | 90.19   | 3673  | 3513    | 3428  | 93.71  | 25.00 |  |  |                      |
| abz6 | 2296  | 2211    | 2189  | 31.18  | 1.83    | 2818  | 2636    | 2472  | 107.60  | 3201  | 2923    | 2745  | 155.40  | 2769  | 2631    | 2527  | 95.66  | 25.00 |  |  |                      |
| abz7 | 4539  | 4411    | 4341  | 59.28  | 15.35   | 6269  | 5842    | 5563  | 226.02  | 9709  | 9369    | 8668  | 316.45  | 6982  | 6404    | 5722  | 393.58 | 75.00 |  |  |                      |
| abz8 | 4545  | 4495    | 4466  | 32.01  | 15.81   | 6385  | 6082    | 5870  | 189.18  | 10146 | 9627    | 9074  | 392.67  | 7064  | 6770    | 6321  | 256.68 | 75.00 |  |  |                      |
| abz9 | 4457  | 4397    | 4373  | 24.80  | 16.74   | 6973  | 6453    | 6144  | 277.43  | 10829 | 10335   | 9700  | 410.88  | 8110  | 7276    | 6588  | 415.90 | 75.00 |  |  |                      |
| la30 | 10726 | 10584   | 10495 | 88.13  | 6.42    | 13040 | 12401   | 11821 | 405.55  | 16019 | 14956   | 13811 | 589.83  | 12068 | 11494   | 10983 | 285.37 | 50.00 |  |  |                      |
| la31 | 29392 | 29018   | 28462 | 242.46 | 14.13   | 35215 | 32846   | 31770 | 1044.02 | 45290 | 43115   | 41202 | 1427.34 | 33861 | 32779   | 32245 | 510.85 | 75.00 |  |  |                      |
| la32 | 35503 | 34457   | 33634 | 472.79 | 12.80   | 38189 | 36946   | 35170 | 839.69  | 49773 | 45994   | 43631 | 1897.34 | 37263 | 36196   | 35560 | 529.88 | 75.00 |  |  |                      |
| la33 | 29722 | 29151   | 28471 | 378.48 | 12.33   | 34049 | 33024   | 31594 | 721.81  | 42985 | 42985   | 41358 | 1079.82 | 34047 | 33214   | 32477 | 550.68 | 75.00 |  |  |                      |
| la34 | 33744 | 32570   | 31695 | 359.75 | 12.19   | 35396 | 33832   | 32030 | 1055.38 | 47763 | 45159   | 42825 | 1708.80 | 36400 | 35357   | 34617 | 584.03 | 75.00 |  |  |                      |
| la35 | 35377 | 34308   | 33394 | 525.20 | 12.26   | 37564 | 36975   | 35335 | 727.55  | 50913 | 49209   | 46464 | 1615.36 | 39079 | 38442   | 36570 | 830.94 | 75.00 |  |  |                      |
| mt06 | 89    | 79      | 74    | 4.69   | 0.69    | 91    | 81      | 74    | 4.99    | 91    | 79      | 74    | 5.35    | 88    | 81      | 75    | 4.94   | 15.00 |  |  |                      |
| mt10 | 2049  | 2028    | 2012  | 11.27  | 1.50    | 2515  | 2360    | 2281  | 63.00   | 2597  | 2500    | 2270  | 94.73   | 2615  | 2452    | 2250  | 118.38 | 15.00 |  |  |                      |
| mt20 | 13591 | 13359   | 13037 | 183.99 | 11.31   | 15312 | 14631   | 13965 | 405.81  | 16183 | 14877   | 13027 | 960.90  | 13928 | 13468   | 13063 | 311.52 | 75.00 |  |  |                      |
| Mk01 | 146   | 137     | 130   | 5.12   | 0.42    | 161   | 146     | 140   | 6.52    | 148   | 140     | 136   | 4.33    | 154   | 144     | 141   | 3.97   | 9.00  |  |  |                      |
| Mk05 | 1773  | 1698    | 1615  | 51.67  | 1.90    | 2040  | 1964    | 1842  | 68.24   | 2012  | 1876    | 1806  | 66.50   | 1732  | 1703    | 1670  | 24.18  | 25.00 |  |  |                      |
| Mk10 | 1591  | 1581    | 1557  | 11.17  | 1.56    | 1678  | 1593    | 1498  | 61.80   | 1894  | 1803    | 1744  | 45.65   | 1527  | 1479    | 1450  | 28.03  | 25.00 |  |  |                      |

<sup>a</sup> The test time of the GA, PSO and DE is same.

**Table 9**  
IDDQN compare with GP.

| Data | IDDQN |         |       | GP     |         |       |
|------|-------|---------|-------|--------|---------|-------|
|      | Max   | Average | Min   | Max    | Average | Min   |
| abz5 | 3158  | 3125    | 3092  | 9978   | 8412    | 7109  |
| abz6 | 2296  | 2211    | 2189  | 10541  | 9512    | 8539  |
| abz7 | 4539  | 4411    | 4341  | 28893  | 25831   | 24082 |
| abz8 | 4545  | 4495    | 4466  | 33120  | 28695   | 25055 |
| abz9 | 4457  | 4397    | 4373  | 28763  | 25654   | 22578 |
| la30 | 10726 | 10584   | 10495 | 30992  | 27355   | 25455 |
| la31 | 29392 | 29018   | 28462 | 90794  | 77121   | 67517 |
| la32 | 35503 | 34457   | 33634 | 91758  | 83147   | 73140 |
| la33 | 29722 | 29151   | 28471 | 98103  | 82284   | 74994 |
| la34 | 32228 | 31536   | 31002 | 108650 | 85018   | 75379 |
| la35 | 35377 | 34308   | 33394 | 130627 | 100326  | 86349 |
| mt06 | 89    | 79      | 74    | 157    | 107     | 64    |
| mt10 | 2049  | 2028    | 2012  | 7049   | 6625    | 6162  |
| mt20 | 13591 | 13359   | 13037 | 19917  | 18657   | 16263 |
| Mk01 | 146   | 137     | 130   | 261    | 213     | 178   |
| Mk05 | 1773  | 1689    | 1615  | 2950   | 1953    | 1411  |
| Mk10 | 1591  | 1581    | 1557  | 3250   | 2396    | 1578  |

perience pool prevents the algorithm from succumbing to local optima, thereby enhancing algorithm iteration training efficiency. The initialization of our framework utilized benchmarks from the OR-Library in a static environment while accommodating random machine breakdown in a dynamic context. Experimental results substantiate the superior scheduling outcomes of our algorithm compared to dispatch rules across all instances. Superior performance can still be achieved compared to traditional DDQN, even with significantly reduced training times. Furthermore, the proposed IDDQN algorithm outperforms GA, PSO and DE in most instances, demonstrating its efficacy in solving flexible job shop scheduling problems.

Future research endeavors will focus on various dimensions. Primarily, the artificial feature engineering incorporated in the state space of this study is meticulously crafted to optimize total tardiness in DFJSP. Subsequent investigations will scrutinize alternative methodologies for designing the state space, exploring approaches for the algorithm to exhibit improved generalization across diverse scheduling problems. The design of agents' actions is a significant focus of research. Future studies will prioritize more dexterous actions and may incorporate designs that automatically enable intelligent agents to learn these actions. Furthermore, this study exclusively addresses machine breakdown within DFJSP. However, dynamic disturbances arise from many impact factors in natural production environments. Hence, we aspire to apply the proposed DRL framework to address multi-objective DFJSP challenges.

## Funding

This work was jointly supported by the following projects: The National Natural Science Foundation of China (51805152, 52075401, 52475528), the Scientific Research Foundation for High-level Talents of Hubei University of Technology (GCRC2020009), Zhejiang Province Postdoctoral Optimal Funding Project (ZJ2024148) and the Doctoral Scientific Research Foundation of Hubei University of Technology (BSQD2020007).

## CRedit authorship contribution statement

**Rui Wu:** Project administration, Funding acquisition, Conceptualization, Methodology; **Jianxin Zheng:** Data curation, Visualization, Software, Writing - original draft; **Xixing Li:** Methodology, Writing - review & editing, Data curation, Funding acquisition; **Hongtao Tang:** Validation, Writing - review & editing, Funding acquisition; **Xi Vincent Wang:** Visualization, Writing - review & editing, Supervision; **Yibing Li:** Supervision, Writing - review & editing.

## Data availability

Data will be made available on request.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

The authors would like to thank the reviewers for their insightful comments and useful suggestions.

## References

- Adams, J., Balas, E., & Zawack, D. (1988). The shifting bottleneck procedure for job shop scheduling. *Management Science*, 34(3), 391–401. <https://doi.org/10.1287/mnsc.34.3.391>
- Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3), 157–183. <https://doi.org/10.1007/BF02023073>
- Branke, J., Nguyen, S., Pickardt, C. W., & Zhang, M. (2016). Automated design of production scheduling heuristics: A review. *IEEE Transactions on Evolutionary Computation*, 20(1), 110–124. <https://doi.org/10.1109/TEVC.2015.2429314>
- Chen, X., Bai, R., Qu, R., Dong, J., & Jin, Y. (2024). Deep reinforcement learning assisted genetic programming ensemble hyper-heuristics for dynamic scheduling of container port trucks. *IEEE Transactions on Evolutionary Computation*, (pp. 1). <https://doi.org/10.1109/TEVC.2024.3381042>
- Das, K. (2008). A comparative study of exponential distribution vs weibull distribution in machine reliability analysis in a CMS design. *Computers & Industrial Engineering*, 54(1), 12–33. <https://doi.org/10.1016/j.cie.2007.06.030>
- Dauzère-Pérès, S., Ding, J., Shen, L., & Tamssaouet, K. (2023). The flexible job shop scheduling problem: A review. *European Journal of Operational Research*, (p. S037722172300382X). <https://doi.org/10.1016/j.ejor.2023.05.017>
- Destouet, C., Tlahig, H., Bettayeb, B., & Mazari, B. (2023). Flexible job shop scheduling problem under Industry 5.0: A survey on human reintegration, environmental consideration and resilience improvement. *Journal of Manufacturing Systems*, 67, 155–173. <https://doi.org/10.1016/j.jmsy.2023.01.004>
- Doh, H.-H., Yu, J.-M., Kim, J.-S., Lee, D.-H., & Nam, S.-H. (2013). A priority scheduling approach for flexible job shops with multiple process plans. *International Journal of Production Research*, 51(12), 3748–3764. <https://doi.org/10.1080/00207543.2013.765074>
- Fan, J., Zhang, C., Liu, Q., Shen, W., & Gao, L. (2022). An improved genetic algorithm for flexible job shop scheduling problem considering reconfigurable machine tools with limited auxiliary modules. *Journal of Manufacturing Systems*, 62, 650–667. <https://doi.org/10.1016/j.jmsy.2022.01.014>
- Friedman, L., & Komogortsev, O. V. (2019). Assessment of the effectiveness of seven biometric feature normalization techniques. *IEEE Transactions on Information Forensics and Security*, 14(10), 2528–2536. <https://doi.org/10.1109/TIFS.2019.2904844>
- Fu, J., Yao, W., Sun, G., Ma, Z., Dong, B., Ding, J., & Wu, L. (2024). Multirobot cooperative path optimization approach for multiobjective coverage in a congestion risk environment. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(3), 1816–1827. <https://doi.org/10.1109/TSMC.2023.3329970>
- Gan, X., Zuo, Y., Yang, G., Zhang, A., & Tao, F. (2024). Dynamic scheduling for dual-objective job shop with machine breakdown by reinforcement learning. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, 238(1–2), 3–17. <https://doi.org/10.1177/095444052431167086>
- Gui, Y., Tang, D., Zhu, H., Zhang, Y., & Zhang, Z. (2023). Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. *Computers & Industrial Engineering*, 180, 109255. <https://doi.org/10.1016/j.cie.2023.109255>
- Guo, H., Liu, J., Wang, Y., & Zhuang, C. (2024a). An improved genetic programming hyper-heuristic for the dynamic flexible job shop scheduling problem with reconfigurable manufacturing cells. *Journal of Manufacturing Systems*, 74, 252–263. <https://doi.org/10.1016/j.jmsy.2024.03.009>
- Guo, H., Ma, Y., Ma, Z., Chen, J., Zhang, X., Cao, Z., Zhang, J., & Gong, Y.-J. (2024b). Deep reinforcement learning for dynamic algorithm selection: A proof-of-principle study on differential evolution. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, (pp. 1–13). <https://doi.org/10.1109/TSMC.2024.3374889>
- Fisher, H. (1963). Probabilistic learning combinations of local job-shop scheduling rules. *Industrial Scheduling*.
- Hammami, N. E. H., Lardeux, B., Atidel, B., & Jridi, M. (2024). Design and calibration of a DRL algorithm for solving the job shop scheduling problem under unexpected job arrivals. *Flexible Services and Manufacturing Journal*. <https://doi.org/10.1007/s10696-024-09540-2>
- Han, W., Guo, F., & Su, X. (2019). A reinforcement learning method for a hybrid flow-shop scheduling problem. *Algorithms*, 12(11), 222. <https://doi.org/10.3390/a12110222>
- Jeong, K.-C., & Kim, Y. D. (1998). A real-time scheduling mechanism for a flexible manufacturing system: Using simulation and dispatching rules. *International Journal of Production Research*, 36(9), 2609–2626. <https://doi.org/10.1080/002075498192733>
- Lei, K., Guo, P., Zhao, W., Wang, Y., Qian, L., Meng, X., & Tang, L. (2022). A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem. *Expert Systems with Applications*, 205, 117796. <https://doi.org/10.1016/j.eswa.2022.117796>
- Li, H., Assis, K. D. R., Yan, S., & Simeonidou, D. (2022). DRL-Based long-term resource planning for task offloading policies in multiserver edge computing networks. *IEEE Transactions on Network and Service Management*, 19(4), 4151–4164. <https://doi.org/10.1109/TNSM.2022.3191748>
- Li, R., Gong, W., Wang, L., Lu, C., & Dong, C. (2024). Co-evolution with deep reinforcement learning for energy-aware distributed heterogeneous flexible job shop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(1), 201–211. <https://doi.org/10.1109/TSMC.2023.3305541>
- Liao, Z., Gong, W., & Li, S. (2023). Two-stage reinforcement learning-based differential evolution for solving nonlinear equations. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(7), 4279–4290. <https://doi.org/10.1109/TSMC.2023.3245212>
- Lin, J., Zhu, L., & Gao, K. (2020). A genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem. *Expert Systems with Applications*, 140, 112915. <https://doi.org/10.1016/j.eswa.2019.112915>
- Liu, C.-L., & Huang, T.-H. (2023). Dynamic job-shop scheduling problems using graph neural network and deep reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(11), 6836–6848. <https://doi.org/10.1109/TSMC.2023.3287655>
- Liu, X.-F., Fang, Y., Zhan, Z.-H., & Zhang, J. (2023). Strength learning particle swarm optimization for multiobjective multirobot task scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(7), 4052–4063. <https://doi.org/10.1109/TSMC.2023.3239953>
- Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*, 91, 106208. <https://doi.org/10.1016/j.asoc.2020.106208>
- Ma, D. Y., He, C. H., Wang, S. Q., Han, X. M., & Shi, X. H. (2018). Solving fuzzy flexible job shop scheduling problem based on fuzzy satisfaction rate and differential evolution. *Advances in Production Engineering & Management*, 13(1), 44–56. <https://doi.org/10.14743/apem.2018.1.272>
- Nguyen, S., Mei, Y., & Zhang, M. (2017). Genetic programming for production scheduling: A survey with a unified framework. *Complex & Intelligent Systems*, 3(1), 41–66. <https://doi.org/10.1007/s40747-017-0036-x>
- Nouiri, M., Bekrar, A., Jemai, A., Niar, S., & Ammari, A. C. (2018). An effective and distributed particle swarm optimization algorithm for flexible job-shop scheduling problem. *Journal of Intelligent Manufacturing*, 29(3), 603–615. <https://doi.org/10.1007/s10845-015-1039-3>
- Park, J., Mei, Y., Nguyen, S., Chen, G., & Zhang, M. (2018). Investigating a machine breakdown genetic programming approach for dynamic job shop scheduling. In M. Castelli, L. Sekanina, M. Zhang, S. Cagnoni, & P. García-Sánchez (Eds.), *Genetic programming* (pp. 253–270). Cham: Springer International Publishing. [https://doi.org/10.1007/978-3-319-77553-1\\_16](https://doi.org/10.1007/978-3-319-77553-1_16)
- Peng, B., Lü, Z., & Cheng, T. C. E. (2015). A tabu search/path relinking algorithm to solve the job shop scheduling problem. *Computers & Operations Research*, 53, 154–164. <https://doi.org/10.1016/j.cor.2014.08.006>
- Priore, P., Ponte, B., Puente, J., & Gómez, A. (2018). Learning-based scheduling of flexible manufacturing systems using ensemble methods. *Computers & Industrial Engineering*, 126, 282–291. <https://doi.org/10.1016/j.cie.2018.09.034>
- Lawrence, S. (1984). Resource constrained project scheduling : An experimental investigation of heuristic scheduling techniques (supplement). *Graduate School of Industrial Administration, Carnegie-Mellon University*.
- Sahu, S. K., Mokhadde, A., & Bokde, N. D. (2023). An overview of machine learning, deep learning, and reinforcement learning-based techniques in quantitative finance: Recent progress and challenges. *Applied Sciences*, 13(3), 1956. <https://doi.org/10.3390/app13031956>
- Shahrabi, J., Adibi, M. A., & Mahootchi, M. (2017). A reinforcement learning approach to parameter estimation in dynamic job shop scheduling. *Computers & Industrial Engineering*, 110, 75–82. <https://doi.org/10.1016/j.cie.2017.05.026>
- Shakya, A. K., Pillai, G., & Chakrabarty, S. (2023). Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, 231, 120495. <https://doi.org/10.1016/j.eswa.2023.120495>
- Shen, L., Dauzère-Pérès, S., & Neufeld, J. S. (2018). Solving the flexible job shop scheduling problem with sequence-dependent setup times. *European Journal of Operational Research*, 265(2), 503–516. <https://doi.org/10.1016/j.ejor.2017.08.021>
- Shi, D., Fan, W., Xiao, Y., Lin, T., & Xing, C. (2020). Intelligent scheduling of discrete automated production line via deep reinforcement learning. *International Journal of Production Research*, 58(11), 3362–3380. <https://doi.org/10.1080/00207543.2020.1717008>
- Shiue, Y.-R., Lee, K.-C., & Su, C.-T. (2018). Real-time scheduling for a smart factory using a reinforcement learning approach. *Computers & Industrial Engineering*, 125, 604–614. <https://doi.org/10.1016/j.cie.2018.03.039>
- Song, Y., Ou, J., Pedrycz, W., Suganthan, P. N., Wang, X., Xing, L., & Zhang, Y. (2024). Generalized model and deep reinforcement learning-based evolutionary method for multitype satellite observation scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(4), 2576–2589. <https://doi.org/10.1109/TSMC.2023.3345928>
- Wang, H., Sarker, B. R., Li, J., & Li, J. (2021). Adaptive scheduling for assembly job shop with uncertain assembly times based on dual Q-learning. *International Journal of Production Research*, 59(19), 5867–5883. <https://doi.org/10.1080/00207543.2020.1794075>
- Wang, L., Cai, J., Li, M., & Liu, Z. (2017). Flexible job shop scheduling problem using an improved ant colony optimization. *Scientific Programming*, 2017, e9016303. <https://doi.org/10.1155/2017/9016303>

- Wang, X., Wang, S., Liang, X., Zhao, D., Huang, J., Xu, X., Dai, B., & Miao, Q., et al. (2022). Deep reinforcement learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, PP. <https://doi.org/10.1109/TNNLS.2022.3207346>
- Wang, Y.-F. (2020). Adaptive job shop scheduling strategy based on weighted Q-learning algorithm. *Journal of Intelligent Manufacturing*, 31(2), 417–432. <https://doi.org/10.1007/s10845-018-1454-3>
- Wang, Z., & Liao, W. (2023). Smart scheduling of dynamic job shop based on discrete event simulation and deep reinforcement learning. *Journal of Intelligent Manufacturing*. <https://doi.org/10.1007/s10845-023-02161-w>
- Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd international conference on machine learning* (pp. 1995–2003). PMLR.
- Xu, M., Mei, Y., Zhang, F., & Zhang, M. (2024). Niching genetic programming to learn actions for deep reinforcement learning in dynamic flexible scheduling. *IEEE Transactions on Evolutionary Computation*, (pp. 1). <https://doi.org/10.1109/TEVC.2024.3395699>
- Yan, Q., Wang, H., & Wu, F. (2022). Digital twin-enabled dynamic scheduling with preventive maintenance using a double-layer Q-learning algorithm. *Computers & Operations Research*, 144, 105823. <https://doi.org/10.1016/j.cor.2022.105823>
- Yang, Y., Shen, B., & Han, Q.-L. (2024). Dynamic event-triggered scaled consensus of multi-agent systems in reliable and unreliable networks. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(2), 1124–1136. <https://doi.org/10.1109/TSMC.2023.3324142>
- Zhang, F., Nguyen, S., Mei, Y., & Zhang, M. (2021). Genetic programming for production scheduling: An evolutionary learning approach. *Machine learning: Foundations, methodologies, and applications*. Singapore: Springer Singapore. <https://doi.org/10.1007/978-981-16-4859-5>
- Zhang, G., Lu, X., Liu, X., Zhang, L., Wei, S., & Zhang, W. (2022). An effective two-stage algorithm based on convolutional neural network for the bi-objective flexible job shop scheduling problem with machine breakdown. *Expert Systems with Applications*, 203, 117460. <https://doi.org/10.1016/j.eswa.2022.117460>
- Zhang, Y., Jiang, C., & Zhang, P. (2024). Security-aware resource allocation scheme based on DRL in cloud-edge-terminal cooperative vehicular network. *IEEE Internet of Things Journal*, 11(1), 95–104. <https://doi.org/10.1109/JIOT.2023.3293497>
- Zhao, B., Shi, G., & Liu, D. (2023). Event-triggered local control for nonlinear interconnected systems through particle swarm optimization-based adaptive dynamic programming. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(12), 7342–7353. <https://doi.org/10.1109/TSMC.2023.3298065>
- Zhao, L., Fan, J., Zhang, C., Shen, W., & Zhuang, J. (2024). A DRL-based reactive scheduling policy for flexible job shops with random job arrivals. *IEEE Transactions on Automation Science and Engineering*, 21(3), 2912–2923. <https://doi.org/10.1109/TASE.2023.3271666>
- Zhao, X., Zong, Q., Tian, B., Zhang, B., & You, M. (2019). Fast task allocation for heterogeneous unmanned aerial vehicles through reinforcement learning. *Aerospace Science and Technology*, 92, 588–594. <https://doi.org/10.1016/j.ast.2019.06.024>
- Zheng, Y., Lin, L., Zhang, T., Chen, H., Duan, Q., Xu, Y., & Wang, X. (2022). Enabling robust DRL-driven networking systems via teacher-student learning. *IEEE Journal on Selected Areas in Communications*, 40(1), 376–392. <https://doi.org/10.1109/JSAC.2021.3126085>
- Zhu, Z., & Zhao, H. (2022). A survey of deep RL and IL for autonomous driving policy learning. *IEEE Transactions on Intelligent Transportation Systems*, 23(9), 14043–14065. <https://doi.org/10.1109/TITS.2021.3134702>